UNIVERSITÀ
DI SIENA
1240

University of Siena

Department of Information Engineering and Mathematics

PathNet: A* Search for Multilayer Perceptron Training

Kevin Gagliano

Jacopo Andreucci

Supervisor:

Prof. Edmondo trentin

Academic Year 2024-2025

Contents

Chapter 1

The A* Search Algorithm

1.1 Overview and Context

The **A* (A-star) search algorithm** is a cornerstone of Artificial Intelligence and computer science, highly effective for pathfinding and optimal graph traversal. Developed in 1968 by Hart, Nilsson, and Raphael as an extension of Dijkstra's algorithm, A* is classified as an *informed search* or *best-first search* method. Unlike uninformed searches, A* strategically explores the solution space by using estimated knowledge about the goal, enabling it to efficiently find the shortest path between a start and goal node. Its classical applications include video game pathfinding, network routing, and logistical planning.

1.2 The Evaluation Function $f(n)$

The central innovation of the A* algorithm lies in its heuristic evaluation function, $f(n)$, which estimates the total cost of the path from the starting point through the current node n to the goal. This function is defined as the sum of two components: the true cost from the start, $g(n)$, and the estimated cost to the goal, $h(n)$.

$$f(n) = g(n) + h(n)$$

1.2.1 Path Cost ($g(n)$)

The function $g(n)$ represents the actual cost of the path taken from the initial starting node to the current node n. This is a known, objective value and is a direct measure of the work done so far. In traditional pathfinding, $g(n)$ might be the distance traveled;

in optimization problems like the one discussed in this report, $g(n)$ represents the cumulative loss differential between two consecutive node while traversing the search graph.

1.2.2 Heuristic Estimate ($h(n)$)

The function $h(n)$ is the heuristic estimate of the cost from the current node n to the final goal node. A heuristic is an educated guess that guides the search towards the most promising regions of the search space. The effectiveness and properties of the A* algorithm are highly dependent on the quality of this heuristic.

The heuristic $h(n)$ must satisfy the following critical property to guarantee optimality:

- **Admissibility:** A heuristic $h(n)$ is admissible if it never overestimates the true cost to reach the goal, meaning for all nodes n, $h(n) \leq h^*(n)$, where $h^*(n)$ is the true minimum cost from n to the goal.

If an admissible heuristic is used, A* is guaranteed to find the optimal (lowest-cost) path, provided a path exists.

Consistency

The condition of consistency is a stronger requirement than admissibility. A heuristic $h(n)$ is consistent if, for every node n and every successor node n' connected by an edge with cost $c(n, n')$, the estimated cost from n to the goal is less than or equal to the cost of moving to n' plus the estimated cost from n' to the goal.

$$\textbf{Consistency Condition: } h(n) \leq c(n, n') + h(n')$$

Consistency implies admissibility, provided $h(\text{goal}) = 0$. When a consistent heuristic is used, A* is guaranteed to find the optimal path without needing to re-open nodes (re-examine a node already in the Closed Set), because the f-cost along any optimal path is monotonically non-decreasing.

- **Optimality Guarantee:** If the heuristic $h(n)$ is admissible (and edge costs are non-negative), A* is guaranteed to find the optimal (lowest-cost) path. If the heuristic is consistent, A* is also guaranteed to be optimal and more efficient, as it never needs to process a node more than once.

1.3 Algorithm Mechanics

A* operates by iteratively expanding nodes, maintaining two key data structures:

- **Open Set (or Frontier):** A priority queue containing nodes that have been generated but not yet fully explored. Nodes in the open set are prioritized based on their calculated $f(n)$ value.
- **Closed Set:** A set containing nodes that have already been fully expanded. This prevents the algorithm from revisiting and reprocessing the same node multiple times.

The core loop of the A* algorithm is as follows:

1. Initialize the Open Set with the starting node.
2. While the Open Set is not empty:
    - Select the node n from the Open Set with the lowest $f(n)$ value.
    - If n is the goal state, terminate the search.
    - Move n from the Open Set to the Closed Set.
    - Otherwise, generate all successors of n. For each successor s:
        - Calculate the cost of the path to s via n: $g_{\text{new}}(s) = g(n) + \text{cost}(n, s)$.
        - **Path Evaluation and Update:**
            * If s is in the Open Set, check if $g_{\text{new}}(s) < g(s)$. If true, update $g(s)$ to $g_{\text{new}}(s)$, re-calculate $f(s) = g(s) + h(s)$, and update its priority in the Open Set.
            * If s is in the Closed Set, check if $g_{\text{new}}(s) < g(s)$. If true, this means a cheaper path to s has been found. Update $g(s)$ to $g_{\text{new}}(s)$, re-calculate $f(s) = g(s) + h(s)$, and move s back from the Closed Set to the Open Set for re-expansion.
            * If s is not in either set, it is a newly discovered node. Set its parent to n, set $g(s) = g_{\text{new}}(s)$, calculate $f(s) = g(s) + h(s)$, and add it to the Open Set.

1.4 Properties of A*

The A* search algorithm is known for two key theoretical properties:

- **Completeness:** If a solution path exists from the start node to the goal node, A* is guaranteed to find it, provided the branching factor is finite and edge costs are non-negative.
- **Optimality:** A* is guaranteed to find the lowest-cost path if the heuristic function $h(n)$ is admissible. This makes it a preferred method for problems where minimizing the cost (or, in the context of this project, minimizing the training error) is paramount.

The adaptation of A* from a simple pathfinding tool to a machine learning optimization technique, as explored in this report, requires careful consideration of the state space, the definition of the path cost $g(n)$, and the design of an effective, possibly admissible heuristic $h(n)$.

1.5 Tree Search vs. Graph Search

The problem space solved by A* is fundamentally a graph, yet the search strategy can be executed either as a Tree Search or a Graph Search. The difference lies entirely in how the algorithm handles states (nodes) that can be reached via multiple distinct paths, a common occurrence in graphs with cycles or multiple connecting routes.

1.5.1 Tree Search A*

The Tree Search variant treats every path to a state as a unique node in the search tree.

- **Mechanism:** Does not utilize a Closed Set. A node is generated and expanded without checking if an identical state has been reached previously via a different path.
- **Efficiency:** Avoiding the memory overhead of the Closed Set, leads to the redundant expansion of identical states, severely impacting time efficiency, especially in graphs with many cycles or alternative routes.
- **Optimality:** Only guaranteed if and only if the heuristic function $h(n)$ is admissible.

1.5.2 Graph Search A*

The Graph Search variant is the standard implementation of A* because it explicitly tracks visited states to avoid redundant work.

- **Mechanism:** Utilizes both the Open Set and the Closed Set to ensure that, ideally, no state is expanded more than once.
- **Efficiency:** Significantly more time-efficient than Tree Search, as it avoids re-exploring entire subgraphs.
- **Optimality:**
  - If the heuristic $h(n)$ is **consistent**, a node is guaranteed to be reached via the optimal path when it is first placed in the Closed Set. Therefore, no node ever needs to be re-opened.
  - If $h(n)$ is only **admissible**, the optimal path to a state might be discovered after the state has already been placed in the Closed Set. This motivates the re-opening mechanism.

1.5.3 Implementation Choice

In the context of this project, which adapts A* for machine learning optimization, given the experimental nature of the heuristic $h(n)$ and the $g(n)$ cost definition based on training loss, neither strict **admissibility** nor **consistency** can be guaranteed. Therefore, the implementation utilizes the Graph Search A* algorithm, characterized by the explicit use of both the Open Set and the Closed Set. Crucially, the algorithm includes the necessary logic to re-open nodes from the Closed Set if a new, cheaper path to that node is discovered. This ensures the best possible path is found despite the non-ideal properties of the custom heuristic.

Chapter 2

A* Cost Function Adaptation for Training

The successful application of A* to optimization relies entirely on correctly defining the cost components $g(n)$ and $h(n)$ in the context of neural network training.

2.1 Heuristic Function $h(n)$: Current State Loss

In this formulation, the heuristic $h(n)$ directly represents the network's performance at node n and it is defined as the current loss value evaluated on the training data:

$$h(n) = L(n)$$

This heuristic prioritizes configurations that minimize the objective function, guiding the search toward lower-cost states in the weight landscape.

2.2 Path Cost $g(n)$: Accumulated Loss Differential

The cost-to-come, $g(n)$, represents the cumulative cost of the path taken from the initial configuration to the current node n. The cost of each transition (edge) is defined by the improvement or degradation in loss between the parent and the child node n.

The step cost Δg is defined as:

$$\Delta g = L(\text{n}) - L(\text{parent})$$

The total path cost is then updated iteratively:

$$g(n) = g(\text{parent}) + \Delta g$$

By defining Δg in terms of the loss differential, the path cost effectively tracks the total variation in loss encountered during the traversal of the search space.

2.3 Total Evaluation Function $f(n)$

The combined evaluation function $f(n)$ determines the priority of nodes in the search queue:

$$f(n) = g(n) + h(n) = (g(\text{parent}) + (L(n) - L(\text{parent}))) + L(n)$$

By selecting the node with the minimum $f(n)$, the algorithm seeks a path that minimizes both the absolute error of the current state $L(n)$ and the cumulative change in loss required to reach that state $g(n)$.

2.4 Admissibility Analysis

In standard A* search, a heuristic $h(n)$ is admissible if it never overestimates the true cost to reach the goal, i.e., $h(n) \leq h^*(n)$. In this revised framework, the "goal" is implicitly the global minimum of the loss landscape ($L = 0$), and the cost is measured in units of loss differential.

1. **Definition:** Both the heuristic $h(n) = L(n)$ and the true cost $h^*(n)$ are expressed in the same units (loss). The true cost $h^*(n)$ represents the cumulative sum of loss differentials required to reach the global minimum starting from node n.

2. **The Admissibility Challenge:** For $h(n)$ to be admissible, the condition $h(n) \leq h^*(n)$ must hold. Under the current definitions, $h(n) = L(n)$, which is a non-negative value (the loss), while the true cost $h^*(n)$ is the sum of loss differentials: $h^*(n) = \sum_n^{goal} \Delta g$.

   In any scenario where the path toward the goal is monotonically improving, every step cost $\Delta g = L(n') - L(n)$ is strictly negative. Consequently, the true cost-to-go $h^*(n)$ becomes a summation of negative terms, yielding a negative total value. Because $L(n) \geq 0$, the inequality $L(n) \leq h^*(n)$ is fundamentally violated in every improving step of the training process.

This implies that the heuristic is never admissible during a standard training descent. It provides a positive value (L) for a trajectory that accumulation a negative total cost ($\sum \Delta g$). Admissibility could only be restored if the search moved "uphill" into regions of significantly higher loss; however, since the primary goal is loss minimization, this violation is a permanent structural feature of the search design rather than an error.

2.5 Consistency Analysis

A heuristic is consistent if $h(n) \leq c(n, n') + h(n')$, where $c(n, n')$ is the cost of the transition. With our new definitions:

- $h(n) = L(n)$
- $c(n, n') = \Delta g = L(n') - L(n)$
- $h(n') = L(n')$

Substituting these into the consistency inequality:

$$L(n) \leq (L(n') - L(n)) + L(n') \implies L(n) \leq 2L(n') - L(n) \implies 2L(n) \leq 2L(n')$$

Simplifying this, the **Consistency Condition** becomes:

$$L(n) \leq L(n')$$

Analysis of Scenarios w.r.t. Consistency

Unlike the previous version, consistency is now tied directly to the direction of the loss change ΔL:

1. **Scenario 1: Loss Decrease ($\Delta L < 0$):** When the sliding window update improves the model, $L(n') < L(n)$. In this case, the consistency condition $L(n) \leq L(n')$ is **violated**. This is a mathematically interesting result: every "good" step in training (one that reduces loss) technically violates the A* consistency property under these definitions.

2. **Scenario 2: Loss Increase ($\Delta L > 0$):** If the update moves into a worse region of the landscape, $L(n') > L(n)$. Here, the consistency condition is **satisfied**. This implies that the search is only "consistent" when it is moving away from the solution.

3. **Scenario 3: Stationary Loss ($\Delta L = 0$):** In plateaus where $L(n') = L(n)$, the heuristic is perfectly consistent.

Chapter 3

The Multilayer Perceptron and Traditional Training Methods

3.1 Multilayer Perceptron (MLP) Architecture

The Multilayer Perceptron (MLP) is a fundamental class of feed-forward artificial neural networks. It is distinguished by having one or more *hidden layers* between the input layer and the output layer, which allows the network to learn complex, non-linear relationships between inputs and outputs.

3.1.1 Neuron Structure and Function

The basic unit of the MLP is the artificial neuron, which receives a set of inputs, each associated with a synaptic weight. The neuron performs two main operations:

1. **Weighted Aggregation:** It calculates the weighted sum of the inputs, to which a *bias* (β) is added:

$$z = \left(\sum_{i=1}^{m} w_i x_i \right) + \beta$$

   where x_i are the inputs, w_i are the weights, and m is the number of inputs.

2. **Activation Function:** It applies a non-linear activation function (σ) to the aggregated result to produce the neuron's output (a):

$$a = \sigma(z)$$

3.1.2 The Loss Function

Training an MLP is an optimization problem that aims to minimize the error between the network's predicted output and the desired target output. This error is quantified by the loss function E.

3.2 The Standard Training Method: Backpropagation

Traditionally, the MLP is trained using the **Backpropagation** algorithm, which utilizes the gradient of the loss function to update the network's weights.

3.2.1 Theoretical Basis: Gradient Descent

The most basic iterative algorithm for weight adjustment is *Gradient Descent*. It moves the weights in the direction of the steepest descent of the cost function:

$$w_{t+1} = w_t - \eta \cdot g_t$$

where η is the *learning rate* and $g_t = \nabla E(w_t)$. While this "Stochastic" Gradient Descent (SGD) forms the historical basis of neural network training, it often suffers from slow convergence and sensitivity to the choice of η.

3.3 Selected Gradient-Based Benchmark: The Adam Optimizer

In this study, rather than utilizing standard SGD, we employ the **Adam (Adaptive Moment Estimation)** optimizer as the representative gradient-based benchmark. Adam is chosen due to its status as the current state-of-the-art for training deep architectures, providing a more robust and efficient baseline for comparison against the proposed A* method.

3.3.1 Mathematical Formulation

Adam optimizes training by maintaining moving averages of both the gradients m_t and the squared gradients v_t:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

To ensure the estimates are unbiased during the initial steps, bias-correction is applied:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

The resulting update rule is:

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

This adaptive mechanism allows for a more reliable descent in complex loss landscapes compared to fixed-rate methods.

3.4 Motivation for the A* Approach

Despite the sophistication of adaptive optimizers like Adam, gradient-based methods possess intrinsic limitations that motivate the exploration of informed search algorithms like A*:

- **Local Minima and Saddle Points:** Even with the momentum and scaling features of Adam, gradient-based methods are susceptible to stalling in saddle points or becoming trapped in local minima where the local gradient is zero.
- **Local Nature:** Adam remains a *local* optimizer. It relies entirely on the derivative at the current point w_t and its immediate history. It lacks the global "vision" or heuristic foresight of the cost surface that an informed search algorithm like A* can provide by exploring multiple potential paths in the weight space.

This study proposes to frame the MLP training not as a continuous descent problem, but as a pathfinding problem in a quantized weight space. By utilizing a heuristic function $h(n)$ to estimate the distance to the global minimum, A* can potentially navigate the weight space with greater global awareness than the backpropagation-based methods described in this chapter.

Chapter 4

The A* Search Framework for MLP Training

The primary objective of this project is to reformulate the optimization challenge of training a Multilayer Perceptron (MLP) as a **shortest-path problem** within a discrete, high-dimensional weight space. This chapter introduces the framework designed to achieve this, detailing how the continuous weight space is discretized, how the nodes and edges of the search graph are defined, how the challenge of main memory saturation is managed and how the A* algorithm's is adapted to guide the network towards optimal weights.

4.1 Discretization of the Weight Space

The A* algorithm fundamentally operates on a discrete graph. Therefore, the continuous domain of real-valued weight parameters must be converted into a finite, discrete set of possible states. This is handled by the `QuantizedMLP` class.

4.1.1 Quantized State Definition

To define the search nodes, the floating-point parameters (weights and biases) of the MLP are subjected to uniform quantization. Given a predefined **quantization factor** (Q), every weight w_i is rounded to the nearest multiple of $1/Q$.

$$w_{i,\text{quantized}} = \text{round}(w_i \cdot Q)/Q$$

The number of potential discrete weight configurations defines the size of the

search space. To manage the complexity and avoid instability, the parameters are constrained to a defined numerical range [min_range, max_range].

The total number of possible states is:

$$[\mathrm{Q}(\mathrm{min\_range} + \mathrm{max\_range}) + 1]^{N_{\mathrm{params}}}$$

4.1.2 State Representation and Hashing

In the A* implementation, the efficacy of the algorithm relies on quickly checking if a particular weight configuration has been visited before, or if a better path to that configuration has been found. This requires a unique, hashable identifier for each state.

The `QuantizedMLP` utilizes the `get_state_hash()` method, which concatenates all quantized parameters into a single, ordered tuple of integers (by multiplying the quantized weights by the quantization factor Q). This tuple serves as the key in the Closed Set (represented by the `g_costs` dictionary in the implementation) for tracking the minimum cost found so far to reach that state.

4.2 The Search Node

In the context of A* based optimization, a search node n is defined as a discrete point within a high-dimensional graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. Each vertex $v \in \mathcal{V}$ represents a unique, quantized configuration of the neural network's weights and biases. The edges $\mathcal{E}$ are induced by the possible transitions permitted by the sliding window kernel operations, which move the network from one quantized state to another.

A single search node n is encapsulated by the `SearchNode` class, which maintains the following state and metadata:

1. **Quantized State:** The current `QuantizedMLP` configuration, representing a unique coordinate in the quantized parameter space.
2. **Cost-to-come ($g(n)$):** The cumulative cost incurred from the initial configuration to the current node, calculated as the sum of loss differentials along the path.
3. **Heuristic Estimate ($h(n)$):** The estimate of the remaining cost to reach the target loss (the "goal"), here defined by the current loss value $L(n)$.
4. **Total Estimated Cost ($f(n)$):** The evaluation function $f(n) = g(n) + h(n)$, which the A* algorithm uses to prioritize node exploration in the Open List.

5. **Parent Pointer:** A reference to the preceding node in the search graph, allowing for the reconstruction of the optimal training trajectory upon convergence.

4.3 Defining Transitions: Neighborhood Generation Strategies

The edges of the search graph are defined by the allowed transitions between weight configurations, generated by the `get_neighbors` function. We implement a versatile framework supporting three distinct strategies for neighbor generation, each offering a different trade-off between structured and stochastic exploration.

In all methods, a neighbor state s is generated from the current state n by selecting a subset of parameters $\theta \in \text{Layer}_\ell$ and applying a discrete quantization step:

$$\theta_s = \theta_n \pm \frac{1}{Q} \quad \forall \theta \in \text{Sampled Set}$$

4.3.1 Strategy 1: Single-Kernel Transition

This approach utilizes a universal kernel pair ($H \times W$ for weights, $L \times 1$ for biases) applied consistently across all layers. The algorithm dynamically handles architectural constraints through dynamic shrinking: if the target tensor dimensions are smaller than the defined kernel, the window is automatically clipped to fit the available parameter space. This ensures a consistent transition logic while maintaining robustness across heterogeneous layer sizes.

4.3.2 Strategy 2: Layer-Wise Kernel Specification

To allow for higher degrees of freedom, this method supports per-layer customization, while retaining the **dynamic kernel reshaping** mechanism to ensure compatibility with varying tensor dimensions.. For each layer ℓ, a specific weight kernel, bias kernel, and stride pair (x_s, y_s) are defined. This enables the search to adapt its "spatial reach" to the specific role of the layer. For instance, wider kernels may be utilized for initial layers to capture broad features, while narrower kernels can be used for final layers to perform fine-grained adjustments.

$$W^{(l)} = \begin{bmatrix} w_{1,1}^{(l)} & w_{1,2}^{(l)} & w_{1,3}^{(l)} & w_{1,4}^{(l)} & w_{1,5}^{(l)} \\ w_{2,1}^{(l)} & w_{2,2}^{(l)} & w_{2,3}^{(l)} & w_{2,4}^{(l)} & w_{2,5}^{(l)} \\ w_{3,1}^{(l)} & w_{3,2}^{(l)} & w_{3,3}^{(l)} & w_{3,4}^{(l)} & w_{3,5}^{(l)} \\ w_{4,1}^{(l)} & w_{4,2}^{(l)} & w_{4,3}^{(l)} & w_{4,4}^{(l)} & w_{4,5}^{(l)} \end{bmatrix}$$

Figure 4.1: Visualization of a 3×3 kernel at position (1,1) in a quantized weight matrix.

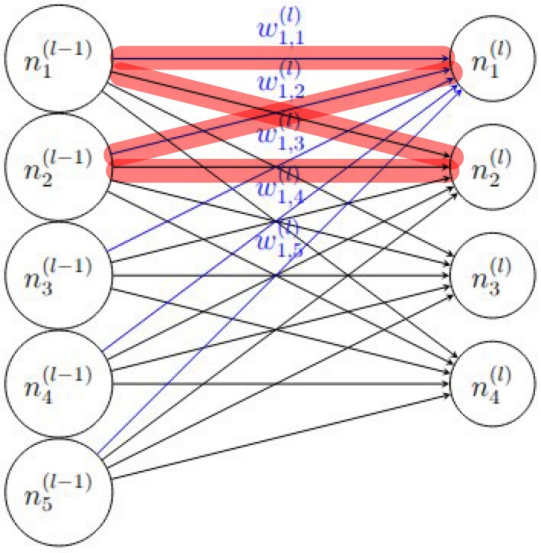

Figure 4.2: Connections within the MLP architecture specifically affected by a localized kernel update.

For both kernel-based strategies, the number of possible transitions (slides) per layer is determined by the valid window positions in an $M \times N$ matrix:

$$\text{Slides}_\ell = \left(\lfloor \frac{M - H}{y_s} \rfloor + 1 \right) \times \left(\lfloor \frac{N - W}{x_s} \rfloor + 1 \right)$$

4.3.3 Dynamic Kernel Reshaping and Scaling

In scenarios where the target network architecture contains layers with parameter tensors smaller than the predefined kernel dimensions, a **dynamic reshaping strategy** is employed. This mechanism allows scaling the kernel size (K) and strides (S) to fit the specific geometry of the parent tensor.

2D Weight Matrices

For a weight matrix of shape (M, N), let the initial weight kernel shape be defined as $\mathbf{k}_w = [k_y, k_x]$. If the matrix dimensions are smaller than the kernel size in any axis ($M < k_y$ or $N < k_x$), the updated kernel $\mathbf{k}'_w = [k'_y, k'_x]$ and strides s'_x, s'_y are calculated as follows:

First, we clip the kernel to the maximum available dimensions:

$$k'_{y,\text{clip}} = \min(M, k_y), \quad k'_{x,\text{clip}} = \min(N, k_x)$$

To maintain a search granularity that is finer than the layer itself, we identify the smaller of these dimensions and reduce it by half:

$$\text{index}_{\min} = \text{argmin}(k'_{y,\text{clip}}, k'_{x,\text{clip}})$$

$$k'_{\text{index}_{\min}} = \max\left(1, \lfloor k'_{\text{index}_{\min},\text{clip}}/2 \rfloor\right)$$

Finally, the traversal strides are set equal to the new kernel dimensions to ensure full, non-overlapping coverage of the search space:

$$s'_x = k'_x, \quad s'_y = k'_y$$

1D Bias Vectors

For bias vectors of length L, the 1D kernel k_b is adjusted if $L < k_b$. The new kernel k'_b and stride s'_y are defined by:

$$k'_b = \max\left(1, \lfloor \min(L, k_b)/2 \rfloor\right)$$

$$s'_y = k'_b$$

This adaptation ensures that even for extremely small layers (such as the final output layer of a regressor), the branching factor remains bounded and the search can still identify discrete updates without requiring manual hyperparameter reconfiguration per layer.

4.3.4 Strategy 3: Random Sampling Neighborhoods

This strategy employs a stochastic approach to define the target set of parameters, governed by two key hyperparameters:

- **Perturbation Ratio (r_p):** Determines the percentage of total parameters within a layer modified in a single transition. For a layer with N_{total} parameters, each neighbor modifies $\lceil r_p \cdot N_{\text{total}} \rceil$ weights selected via random sampling.
- **Search Coverage Ratio (r_c):** Determines the total number of unique neighbors generated for each layer, calculated as $\lceil r_c \cdot N_{\text{total}} \rceil$.

4.3.5 Mathematical Comparison of Branching Factors

The branching factor B represents the number of successor nodes generated from a single parent node. A comparison of the branching factors between the Layer-wise Kernel and Random Sampling methods highlights the difference in search space density:

1. **Kernel-Wise Branching (B_K):** The branching factor is strictly determined by the geometry of the layers and the chosen kernels and strides:

$$B_K = 2 \cdot \sum_{\ell \in \text{Layers}} (\text{Slides}_{\text{weights},\ell} + \text{Slides}_{\text{biases},\ell})$$

2. **Random Sampling Branching (B_R):** The branching factor is proportional to the total parameter count N of the network:

$$B_R = 2 \cdot \sum_{\ell \in \text{Layers}} \lceil r_c \cdot N_\ell \rceil$$

4.3.6 Comparative Analysis: Structured vs. Stochastic Exploration

The three proposed neighborhood generation strategies represent different philosophies in exploring the high-dimensional quantized parameter space. Their effectiveness is evaluated based on four primary criteria: spatial locality, search space pruning, exploration capacity, and hyperparameter complexity.

- **Single-Kernel Method**
  - **Pros:** Provides extreme *search space pruning* by enforcing a uniform transition rule across the entire network. It maximizes *spatial locality* by ensuring that every weight update is part of a cohesive geometric block.
  - **Cons:** Limited *exploration* flexibility; if the network has layers of vastly different sizes (e.g., a massive hidden layer followed by a tiny output layer), the fixed kernel size may be too coarse for some layers and too fine for others.
- **Layer-Wise Kernel Method**
  - **Pros:** Provides an enhanced degree of architectural flexibility while maintaining the inherent advantages of *spatial locality* through localized weight updates. By tailoring kernel sizes and strides to the specific dimensions of each layer, it optimizes the *search space pruning* for the architecture's specific geometry. This prevents "blind spots" in smaller layers while maintaining high-speed traversal in larger ones.
  - **Cons:** Increases the hyperparameter search space. Determining the optimal (WK, BK, S) triplet for every layer adds significant complexity to the pre-training experimental setup.
- **Random Sampling Method**
  - **Pros:** Superior *exploration* capacity. By breaking the constraint of geometric locality, it can discover non-obvious weight correlations and escape local minima where kernel-based movements might stall. It allows for a dense or sparse search controlled entirely by the *perturbation ratio* and *search coverage ratio* parameters.
  - **Cons:** No *spatial locality* is assumed and because updates are stochastic, the search can become "noisy," leading to lower efficiency as the algorithm

may explore many disjoint directions that do not contribute to a cohesive functional change in the network's output.

Table 4.1: Comparison of Neighbor Generation Strategies

Feature	**Single Kernel**	**Layer-Wise**	**Random**
Locality Preservation	High	High	None
Search Space Pruning	High	Moderate	Variable
Exploration Breadth	Low	Moderate	High
Hyperparameter Complexity	Low	High	Low

4.4 Memory Optimization: Beam Search Integration

A significant challenge in applying the A* search algorithm to neural network optimization is the memory-intensive nature of maintaining the *Open* and *Closed* sets. In large-scale architectures, the branching factor B scales with the total number of parameters, leading to an exponential growth of the search tree. Empirical observations showed that without optimization, the search often encountered premature termination due to system RAM saturation. This phenomenon prevented the algorithm from surpassing 100–200 iterations, leaving the network in a sub-optimal state despite remaining potential for loss reduction.

To mitigate this bottleneck, a **Beam Search memory optimization** was designed and integrated into the search framework. This technique effectively caps the memory footprint by limiting the maximum size of the search frontier.

4.4.1 Periodic Set Pruning and Memory Regulation

The optimization is governed by the *Beam Width* (BW) parameter, which defines the target maximum capacity for the *Open* and *Closed* sets. In this implementation, memory regulation is applied periodically rather than at every discrete time step; in our experimental benchmark, this check occurs every 50 iterations.

Consequently, the dimensions of the search sets are not strictly fixed at every step. Between two pruning cycles, the sets are allowed to grow dynamically to accommodate the branching factor of the network. Once the iteration threshold is reached, the algorithm evaluates the current size of the sets. If they exceed the

defined limit, a pruning operation is triggered to restore the memory footprint to the specified BW:

1. **Ranking:** All nodes currently in the set are sorted according to their evaluation function $f(n) = g(n) + h(n)$.
2. **Truncation:** Only the top BW nodes, those representing the most promising search trajectories, are retained.
3. **Memory Release:** All remaining nodes are purged from the main memory, effectively resetting the search frontier to a manageable scale.

This "pulsed" pruning approach strikes a balance between allowing the search to explore diverse local branches in the short term and maintaining long-term stability in the face of hardware memory constraints.

4.4.2 Impact on Training Longevity

The implementation of the Beam Search optimization has demonstrated a profound impact on the scalability of the training process, particularly in deeper architectures. By enforcing a fixed memory bound, the algorithm can navigate the parameter space for significantly longer durations.

- **Scalability:** Networks that previously exhausted memory within 200 iterations can now reliably reach fixed thresholds of $2,000$ iterations or more.
- **Loss Minimization:** The ability to sustain longer search trajectories allows the A* algorithm to escape local plateaus and locate deeper minima in the loss landscape that were previously unreachable due to hardware constraints.
- **Tuning:** The BW parameter acts as a trade-off between search completeness and memory safety. A higher BW allows for a more exhaustive search closer to the original A*, while a lower BW ensures stability in environments with limited hardware resources.

This optimization transforms the A* approach from a memory-bounded search into a scalable iterative optimizer, making it viable for architectures that would otherwise be computationally prohibitive.

4.5 The A* Training Loop Implementation

The core training logic is contained within the `Trainer` class, which manages the A* search:

1. **Initialization:** The process begins by creating an `initial_node` from the starting MLP weights. This node is placed in the priority queue (`open_set`).
2. **Iteration:** In each iteration, the node with the lowest $f(n)$ is retrieved from the `open_set`.
3. **Optimality Check:** Before expanding the node, the current path cost $g(n)$ is checked against the minimum cost previously recorded for that state in `g_costs` to ensure that A* always processes the optimal path to a given state first.
4. **Expansion:** Neighbors are generated by the transition rule ($\pm 1/Q$ change to a weight subset).
5. **Cost Update:** For each neighbor, the new g and h values are calculated. If the new path to the neighbor results in a lower cost-to-come ($g_{\text{new}} < g_{\text{old}}$), the neighbor is either updated in the search space or added to the `open_set`.
6. **Termination:** The search terminates when either the best node retrieved from the `open_set` satisfies the goal condition ($h(n) = 0$) or the maximum number of iterations is reached.

Chapter 5

Benchmark Datasets

In this chapter, we detail the datasets used to evaluate and compare the performance of the A* training framework against standard Stochastic Gradient Descent (SGD). To provide a comprehensive assessment, the benchmarks cover both regression and classification tasks, ranging from synthetic functional mappings to high-dimensional real-world data.

For all experiments, a uniform data partitioning strategy was applied, splitting each dataset into training, validation, and test sets according to an 80-10-10 percentage ratio.

5.1 Regression Datasets

5.1.1 Noisy Sine Function

The Noisy Sine dataset is a synthetic benchmark designed to assess the algorithm's ability to capture non-linear, periodic patterns in a low-dimensional setting. The dataset consists of 1,000 samples generated from a one-dimensional sine wave.

- **Data Generation:** The samples are evenly spaced in the range $x \in [0, 4\pi]$.
- **Noise Profile:** To simulate real-world variance, additive Gaussian noise was introduced: $y = \sin(x) + \epsilon$, where $\epsilon \sim \mathcal{N}(0, 0.1)$.

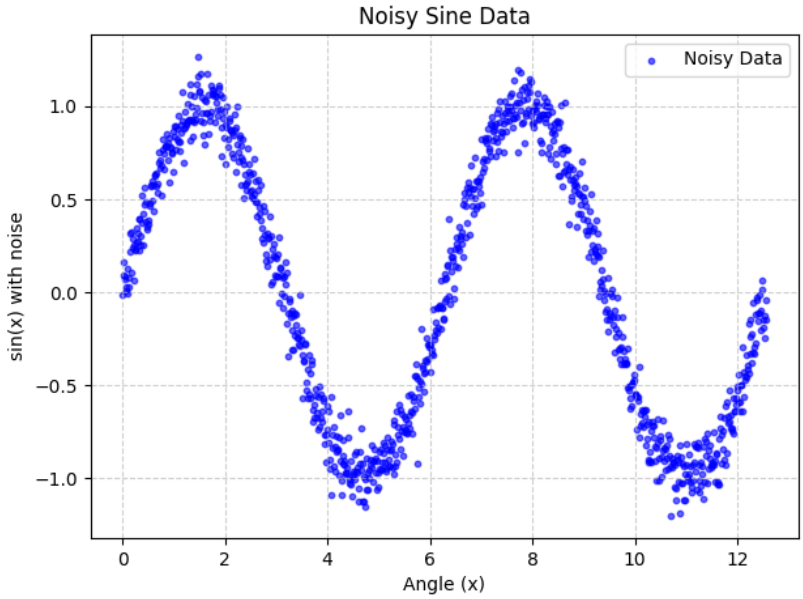

Figure 5.1: Plot of the noisy sine dataset

5.1.2 California Housing

The California Housing dataset provides a high-dimensional, real-world regression challenge. Derived from the 1990 U.S. Census, it contains 20,640 samples, each representing a specific California district.

- **Input Features:** 8 economic and geographic features, including Median Income (MedInc), House Age, Average Rooms, and spatial coordinates (Latitude/Longitude).
- **Target:** The median house value for the district, expressed in units of $100,000.

5.2 Classification Datasets

5.2.1 Iris Flower

The Iris dataset is a classic multiclass classification benchmark. It is a relatively small but well-structured dataset consisting of 150 samples distributed equally across three

species of Iris flowers.

- **Input Features:** 4 physical measurements (sepal length, sepal width, petal length, and petal width).
- **Class Distribution:** Three species: *Setosa*, *Versicolor*, and *Virginica*.

5.2.2 Wine Quality (Red)

The Wine Quality dataset offers a more complex classification task due to its higher dimensionality and larger sample size (1,599 samples) compared to the Iris dataset. It captures the physicochemical properties of Portuguese "Vinho Verde" red wine.

- **Input Features:** 11 physicochemical variables, including fixed acidity, volatile acidity, residual sugar, chlorides, pH, sulphates, and alcohol content.
- **Class Distribution:** The original dataset contains quality scores ranging from 3 to 8. These are shifted to a 0–5 range for modeling.

5.3 Summary of Dataset Characteristics

The table below summarizes the dimensions and objectives of the employed benchmarks.

Table 5.1: Summary of Benchmark Datasets

Dataset	**Type**	**Samples**	**Features**	**Output Classes**
Noisy Sine	Regression	1,000	1	N/A
California Housing	Regression	20,640	8	N/A
Iris Flower	Classification	150	4	3
Wine Quality	Classification	1,599	11	6

Chapter 6

Evaluation Metrics

To rigorously assess the efficacy of the A* search algorithm as a neural network optimizer, we employ two distinct suites of evaluation metrics tailored to regression and classification tasks. For every experiment, performance is measured on the validation and test sets to ensure generalization capability.

Due to the stochastic nature of the initialization and the search trajectories, each experiment is repeated across multiple independent runs. For all metrics defined below, we report the **Mean**, **Standard Deviation**, **Minimum**, and **Maximum** values to capture the stability and potential peak performance of the training process.

6.1 Regression Performance Metrics

In regression tasks, specifically the Noisy Sine and California Housing benchmarks, the objective is to minimize the deviation between the predicted continuous values $\hat{y}$ and the ground-truth values y. We employ four primary metrics:

- **Mean Squared Error (MSE):** Measures the average of the squares of the errors. It is highly sensitive to outliers due to the squaring of the residuals.

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- **Root Mean Squared Error (RMSE):** The square root of the MSE, providing an error metric in the same units as the target variable.

$$\text{RMSE} = \sqrt{\text{MSE}}$$

- **Mean Absolute Error (MAE):** Represents the average of the absolute differences between predictions and actual observations, providing a linear measure of error.

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

- **Coefficient of Determination (R^2):** This metric quantifies the proportion of the variance in the dependent variable (y) that is predictable from the independent variables (x) via the neural network. It acts as a comparison between the model's error and the variance of a simple mean-based baseline.

  The formula is defined as:

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

  Where SS_{res} is the sum of squared residuals (unexplained variance) and SS_{tot} is the total sum of squares (total variance in the data). An R^2 of 1.0 indicates that the model perfectly accounts for all variability in the target data. An R^2 of 0 implies the model is no more informative than the mean of the observed data, while an $R^2 < 0$ indicates that the model provides a worse fit than a horizontal line representing the mean, typically due to a severe overfitting or underfitting.

6.2 Classification Performance Metrics

For classification tasks (Iris and Wine Quality), performance is evaluated based on the model's ability to correctly assign categorical labels. The following metrics are derived from the Confusion Matrix (True Positives TP, True Negatives TN, False Positives FP, and False Negatives FN):

- **Accuracy:** The ratio of correctly predicted observations to the total observations.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision:** The ability of the classifier not to label as positive a sample that is negative.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall (Sensitivity):** The ability of the classifier to find all the positive samples.
$$\text{Recall} = \frac{TP}{TP + FN}$$
- **F1-Score:** The harmonic mean of Precision and Recall, providing a balanced metric particularly useful in the presence of class imbalance.
$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

6.3 Statistical Aggregation

For both regression and classification, the reporting of **Standard Deviation** is critical to our analysis. It serves as a proxy for the *reliability* of the A* search under different kernel and quantization configurations. A low standard deviation across runs indicates that the search logic is consistently finding similar basins of attraction in the weight space, regardless of the random initialization.

Chapter 7

Results Visualization and Statistical Evaluation

To ensure a rigorous comparative analysis between the proposed A^* search framework and the Adam optimizer, we utilize a dual-plotting methodology. This approach allows us to observe not only the average performance but also the reliability and variance of the training processes across multiple independent trials. As Adam represents the state-of-the-art in adaptive gradient-based optimization, it serves as the primary benchmark for evaluating the effectiveness of the A^* approach in the weight space.

7.1 Convergence Analysis: Mean Loss and Variance

The first visualization focuses on the temporal evolution of the training process. By plotting the loss as a function of iterations, we observe the convergence velocity and the stability of the optimization path.

7.1.1 Mean Loss Trajectory ($\bar{L}$)

The primary indicator in this plot is the solid line representing the average loss across N independent runs. This trajectory reveals the global behavior of the optimizer:

- **Convergence Speed:** The slope of the curve indicates how rapidly the algorithm minimizes error.

- **Plateau Level:** The terminal value of the line represents the average expected performance upon reaching $I_{\max}$.

7.1.2 Standard Deviation Shading ($\pm 1\sigma$)

The shaded region surrounding the mean line represents the standard deviation (σ). This area is a critical metric for **algorithmic robustness**:

- **High Consistency (Narrow Band):** A narrow band suggests that the algorithm is insensitive to initial weight configurations and consistently follows a similar optimization path.
- **High Volatility (Wide Band):** A wide band indicates that the search outcome is highly dependent on initialization, suggesting that the algorithm may frequently encounter diverse local minima.

7.2 Distributional Analysis: Final Loss Box Plots

While convergence plots show the "how," box plots provide a definitive "what" regarding the final state of the models. These plots summarize the statistical distribution of the terminal loss values achieved at the end of training.

7.2.1 Component Interpretation

The box plot provides a five-number summary of the terminal results:

- **The Median:** The central horizontal line represents the 50th percentile. This is our primary metric for "typical" performance, as it remains unaffected by isolated training failures.
- **The Interquartile Range (IQR):** The vertical box spans from the 25th (Q_1) to the 75th (Q_3) percentile. A smaller IQR indicates that the algorithm's results are highly clustered.
- **Whiskers:** These represent the spread of the data excluding extreme cases, providing a view of the reliable range of the optimizer.

7.2.2 Formal Identification of Outliers

To maintain statistical integrity, we utilize a standardized method for identifying anomalous runs (outliers). This prevents rare, failed training attempts from skewing the interpretation of the algorithm's general capability.

The length of the whiskers is determined by the IQR, where $IQR = Q_3 - Q_1$. Any data point L_{final} is considered an **outlier** if it falls outside the following boundaries:

$$\text{Lower Boundary} = Q_1 - 1.5 \times IQR$$

$$\text{Upper Boundary} = Q_3 + 1.5 \times IQR$$

In our analysis, outliers appearing above the upper whisker are particularly noteworthy, as they represent runs where the search failed to escape high-loss regions of the parameter space.

Chapter 8

Preliminary Study for Hyperparameters Tuning

The application of the A^* search algorithm to the training of neural networks introduces a unique set of challenges that distinguish it fundamentally from traditional gradient-based optimization. While Gradient Descent operates in a continuous landscape, A^* explores a discretized state-space of weights and biases, where the efficiency of the search is governed by the logic of neighbor generation and the management of computational resources.

This chapter details an extensive experimental campaign—spanning across various network complexities and architectural strategies—aimed at identifying the optimal balance between convergence precision and computational feasibility. Given the exponential growth of the search tree, a "vanilla" implementation of A^* is insufficient for deep architectures. Consequently, this study evaluates several advanced techniques designed to mitigate the "curse of dimensionality" and prevent hardware exhaustion.

The tuning process is structured into four primary investigative areas:

1. **Structural Adaptation:** Testing dynamic versus static kernel reshaping to simplify the architectural search space.

2. **Search Resolution:** Analyzing adaptive quantization factors to allow the algorithm to refine its search as it approaches local minima.

3. **Stochastic Exploration:** A grid search on random sampling parameters to evaluate the effectiveness of non-deterministic neighbor generation.

4. **Resource Optimization:** The integration of Beam Search to transition from an exhaustive memory-heavy search to a resource-constrained optimization process.

By evaluating these components across three distinct network scales, this study establishes a robust hyperparameter baseline that ensures the A^* algorithm can effectively train neural networks within realistic hardware constraints.

8.1 Experimental Setup

To ensure that the results obtained across the various test phases are consistent and comparable, a standardized experimental environment was established. This setup focuses on isolating the effects of hyperparameter changes while keeping the dataset and evaluation metrics constant.

8.1.1 Benchmark Dataset and Network Architectures

The **California Housing Dataset** was selected as the sole benchmark for this tuning phase. This choice provides a regression task of sufficient complexity to highlight the convergence differences between strategies without the overhead of massive image-based datasets. The data distribution consists of 16,512 training samples, 2,064 validation samples, and 2,064 test samples, following a standard $[80\%, 10\%, 10\%]$ split.

To assess how the algorithm scales with parameter count, three architectures were defined:

- **Small Net**: $[8, 32, 16, 1]$ (coarse exploration baseline).
- **Medium Net**: $[8, 64, 32, 16, 1]$ (increased depth and width).
- **Big Net**: $[8, 128, 64, 32, 16, 1]$ (testing memory-intensive limits).

8.1.2 Evaluation Metrics and Baseline Constants

The primary performance indicator is the final average **Mean Squared Error (MSE)** calculated on the training set after 2,000 iterations. Predictive accuracy is further validated through the MSE and R^2 score on the test set.

A set of default parameters, derived from early empirical trials, is maintained as a constant baseline for all tests unless specific variations are required by the experiment logic (see Table 8.1).

Table 8.1: Default hyperparameters and baseline constraints.

Parameter	**Value**	**Parameter**	**Value**
Iterations	2000	Parameter Range	[-10, 10]
Independent Runs	5	Early Stopping Patience	200
Min Quantization Factor	10	Max Quantization Factor	10,000
Loss Improvement Threshold	0.001	Kernels/Stride Decrement	1
Dynamic Logic Patience	100	Quantization Multiplier	10

8.1.3 Categorization of Neighbor Generation Strategies

The most critical aspect of the search algorithm is how it defines its "neighbors" (the next possible configurations of weights). This chapter compares three philosophies:

- **Single Kernel Strategy**: Uses a unified weight/bias kernel that reshapes itself to fit different layers. It represents a low-parameter deterministic approach.
- **Layer-wise Kernels Strategy**: Employs dedicated kernels for each layer, allowing for high structural customization at the cost of more hyperparameters to tune.
- **Random Neighbors Generation**: Introduces stochasticity by selecting a percentage of parameters to perturb (*perturbation ratio*) and generating a specific number of children (*search coverage*).

8.1.4 Specific Tuning Experiments

Following the setup, the chapter is organized into detailed subsections for each experiment. We first examine **Kernel Dynamics**, comparing if dynamic reshaping offers any advantage over fixed kernels. We then transition to the **Quantization Factor Study**, which tests the hypothesis that increasing search resolution during stalls helps the algorithm escape plateaus.

Subsequently, a comprehensive **Grid Search** is performed on Random Sampling parameters to find the ideal balance between perturbation intensity and search breadth. Finally, we conclude with a critical analysis of **Beam Search Optimization**, where we evaluate how limiting the "beam width" affects training network-wide, specifically looking at how it prevents the system-level memory failures inherent in standard A^* searches.

8.2 Dynamic Kernels Reshaping Results

This section presents the experimental results obtained by applying the dynamic kernel reshaping technique across three different network architectures. The primary objective of this evaluation is to compare the performance of dynamically reshaped kernels against statically defined ones.

Specifically, this test seeks to determine whether the dynamic approach offers superior performance or, at minimum, achieves comparable results to the static counterpart. An equivalent performance would be a significant finding, as it would simplify the architectural design process by reducing the necessity for manual selection of optimal kernel dimensions.

8.2.1 Small Net Results

The configuration parameters for the dynamic and static tests are outlined here:

- **Weight Kernel Size:** Max $[4, 4]$, Min $[2, 2]$ (decrement: 1).
- **Bias Kernel Size:** Max [4], Min [2] (decrement: 1).
- **Strides:** Max 4, Min 2.
- **Runs:** 5 for each configuration.

Visual Analysis of Training and Stability

The first point of analysis concerns the training trajectory. As illustrated in Figure 8.1, all models benefit from an initial rapid loss reduction. However, a significant divergence occurs around iteration 250.

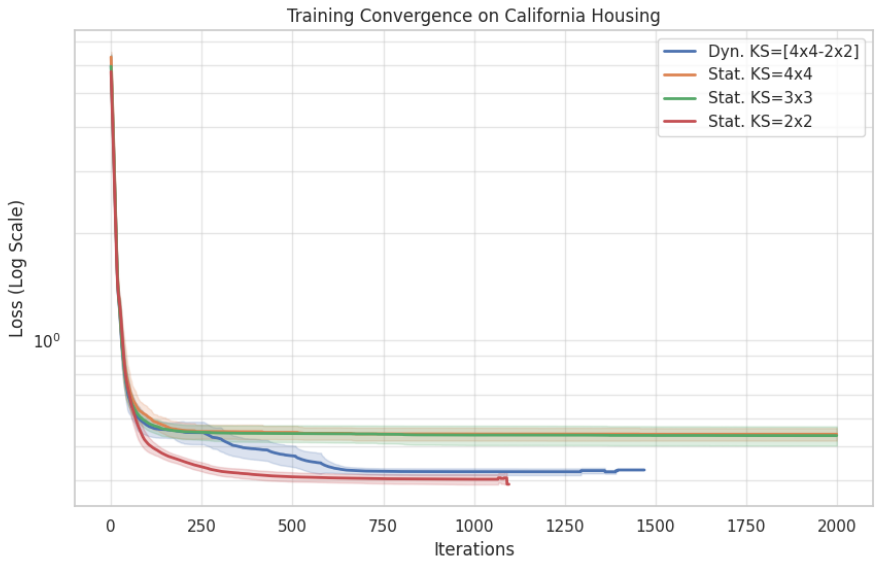

Figure 8.1: Training convergence comparison on a logarithmic scale.

The *Dynamic* approach (blue line) demonstrates a superior ability to adapt compared to larger static kernels (4×4 and 3×3), which plateau prematurely. The dynamic reshaping allows the network to "escape" these local minima, eventually settling into a performance range very close to the optimal 2×2 static configuration.

To assess the reliability of these results, Figure 8.2 displays the distribution of the final loss values across the 5 independent runs.

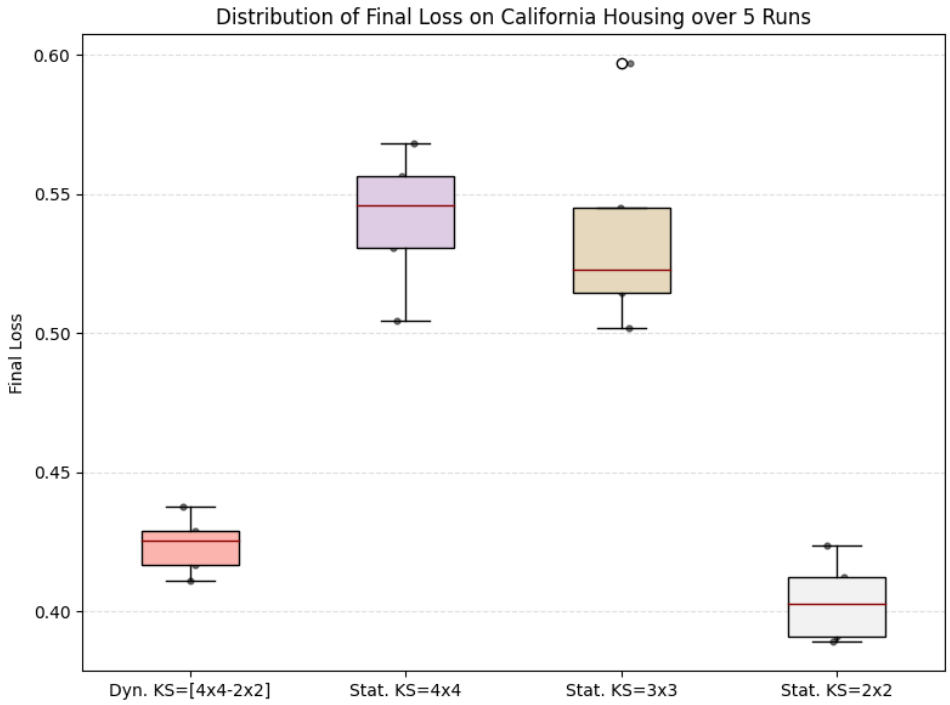

Figure 8.2: Distribution of Final Loss over 5 runs.

The boxplot highlights the stability of the *Dynamic* method, which maintains a very tight distribution. In contrast, the static 3×3 and 4×4 kernels show not only higher average loss but also greater variance and presence of outliers, suggesting a higher sensitivity to weight initialization.

Finally, we examine the broader predictive performance through the distribution of regression metrics (MSE, RMSE, MAE, and R^2), shown in Figure 8.3.

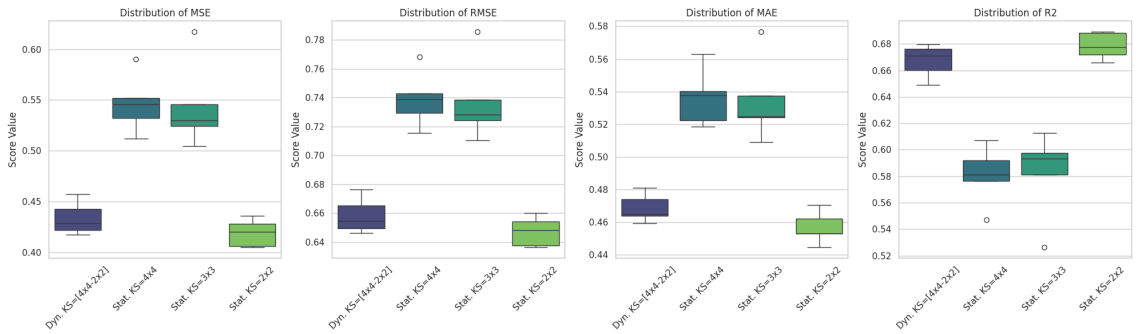

Figure 8.3: Boxplots for MSE, RMSE, MAE, and R^2 across all tested configurations.

The metrics confirm that the Dynamic approach consistently matches or exceeds the performance of large static kernels across all indicators. Notably, the R^2 distribution for the dynamic method is centered around 0.67, proving that it can autonomously find a near-optimal configuration for the California Housing task.

Summary Statistics

The numerical data summarized in Table 8.2 provides the final confirmation of these observations, illustrating the average performance across all metrics.

Metric (Avg)	**Dynamic [4x4-2x2]**	**Static 4x4**	**Static 3x3**	**Static 2x2**
Final Loss	0.4239	0.5411	0.5362	0.4036
Time (s)	736.10	387.30	527.97	721.80
MSE	0.4337	0.5464	0.5444	0.4190
RMSE	0.6585	0.7390	0.7374	0.6473
MAE	0.4686	0.5363	0.5344	0.4567
R^2	0.6673	0.5808	0.5823	0.6785

Table 8.2: Average performance summary for Small Net.

8.2.2 Medium Net Results

The configuration parameters for the dynamic and static tests are outlined here:

- **Weight Kernel Size:** Max $[6, 6]$, Min $[2, 2]$ (decrement: 1).
- **Bias Kernel Size:** Max [6], Min [2] (decrement: 1).
- **Strides:** Max 6, Min 2.
- **Runs:** 5 for each configuration.

Visual Analysis of Training and Stability

As illustrated in Figure 8.4, the training trajectory for the Medium Net shows a clear advantage for the dynamic approach. While large static kernels (6×6 and 5×5) suffer from significant plateaus early in the training, the dynamic reshaping allows the model to continuously refine its representation.

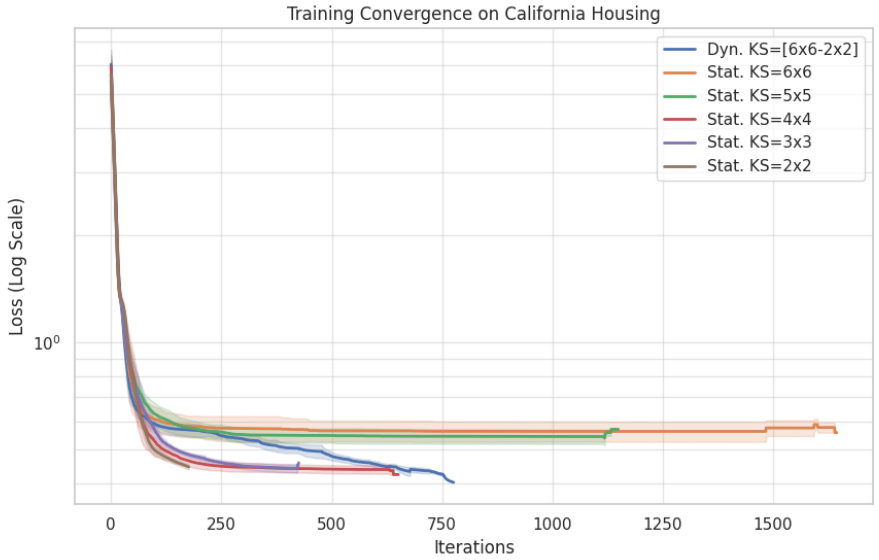

Figure 8.4: Training convergence comparison for the Medium Net on a logarithmic scale.

The *Dynamic* approach (blue line) begins to diverge from the suboptimal larger kernels after approximately 150 iterations, eventually achieving a lower final loss than even the best-performing static configurations (4×4 and 3×3).

To assess the reliability of these results, Figure 8.5 displays the distribution of the final loss values across the 5 independent runs.

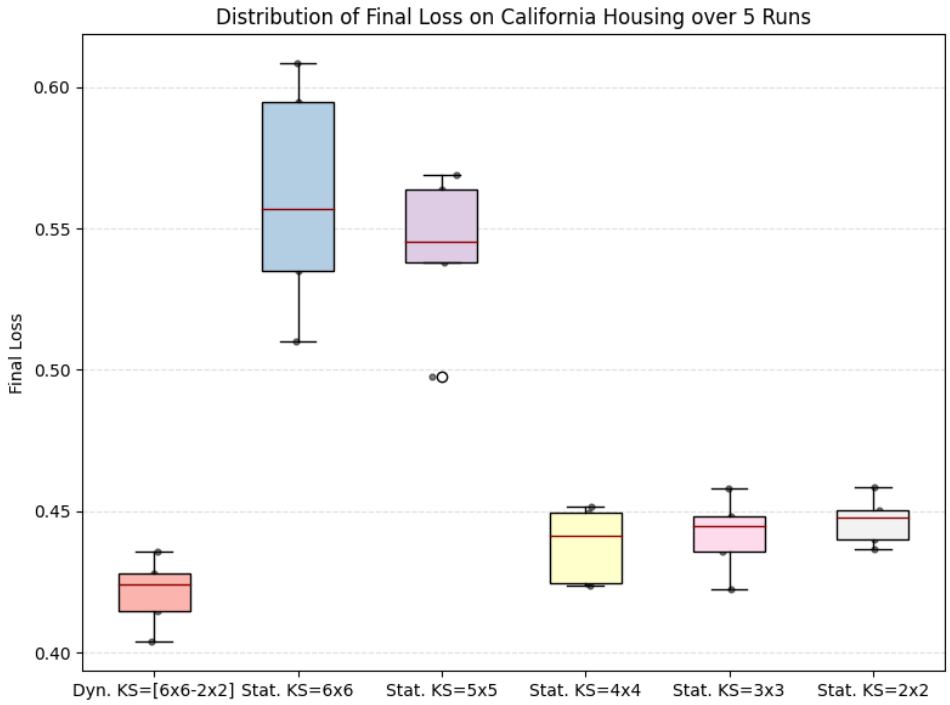

Figure 8.5: Distribution of Final Loss over 5 runs for the Medium Net.

The boxplot confirms that the *Dynamic* method is the most stable and effective configuration for this architecture. It exhibits the lowest median loss and a remarkably tight distribution, whereas the larger static kernels (6×6 and 5×5) show both poor performance and higher variance.

Finally, we examine the broader predictive performance through the distribution of regression metrics (MSE, RMSE, MAE, and R^2), shown in Figure 8.6.

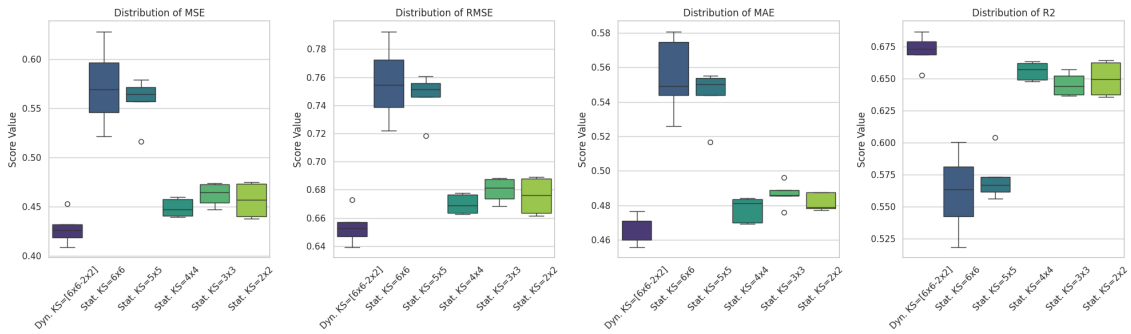

Figure 8.6: Boxplots for MSE, RMSE, MAE, and R^2 across all Medium Net configurations.

The metrics highlight a critical finding: for the Medium Net, the Dynamic approach consistently outperforms all static kernels across every indicator. Notably, the R^2 distribution for the dynamic method is centered above 0.67, while all static configurations, including the previously optimal smaller kernels, fail to reach the same level of predictive accuracy.

Summary Statistics

The numerical data in Table 8.3 provides the final confirmation. The Dynamic approach achieves the best average R^2 (0.6720) and the lowest average MSE (0.4275), proving its superiority in more complex architectures.

Metric (Avg)	**Dyn. [6x6-2x2]**	**Stat. 6x6**	**Stat. 5x5**	**Stat. 4x4**	**Stat. 3x3**	**Stat. 2x2**
Final Loss	0.4212	0.5610	0.5427	0.4382	0.4419	0.4467
Time (s)	810.86	745.08	745.80	751.14	758.06	756.91
MSE	0.4275	0.5720	0.5574	0.4487	0.4622	0.4565
RMSE	0.6537	0.7559	0.7465	0.6698	0.6798	0.6756
MAE	0.4670	0.5549	0.5440	0.4777	0.4866	0.4820
R^2	0.6720	0.5611	0.5723	0.6557	0.6454	0.6497

Table 8.3: Average performance summary for the Medium Net architecture.

8.2.3 Big Net Results

The Big Net architecture represents the most complex scenario, evaluated with the following setup:

- **Weight Kernel Size:** Max $[8, 8]$, Min $[3, 3]$ (decrement: 1).
- **Bias Kernel Size:** Max [8], Min [3] (decrement: 1).
- **Strides:** Max 8, Min 3.
- **Runs:** 5 for each configuration.

Visual Analysis of Training and Stability

The training trajectory for the Big Net, shown in Figure 8.7, illustrates the difficulty of optimizing larger architectures. We observe a significant failure in the *Static 3x3* configuration, which remains trapped in a high-loss state.

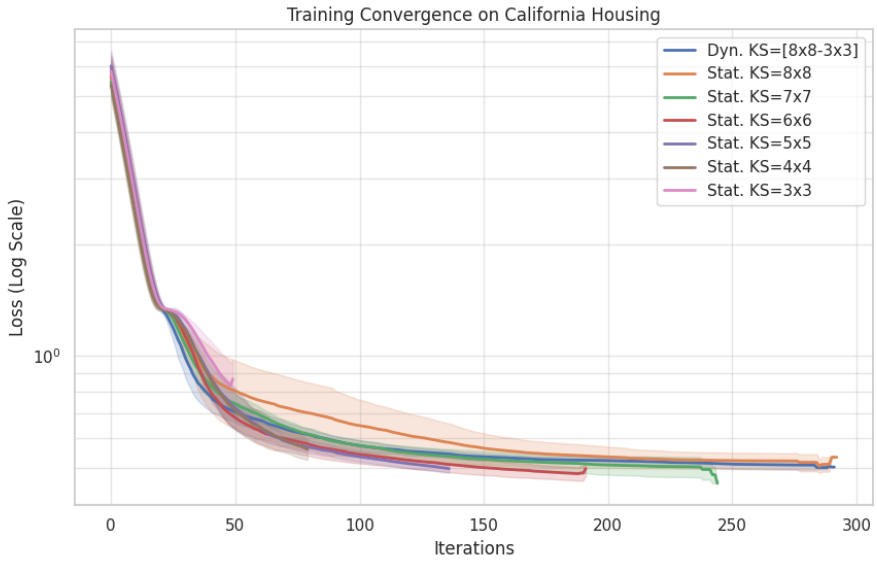

Figure 8.7: Training convergence comparison for the Big Net on a logarithmic scale.

The *Dynamic* approach (blue line) successfully avoids the initialization issues that plague the smallest static kernels. It follows a stable descent path, mirroring the behavior of the most effective static kernels (6×6 and 7×7), and proves that

starting with a larger receptive field before reshaping is beneficial for deeper or wider networks.

Figure 8.8 highlights the distribution of final loss values, providing insights into the robustness of each method.

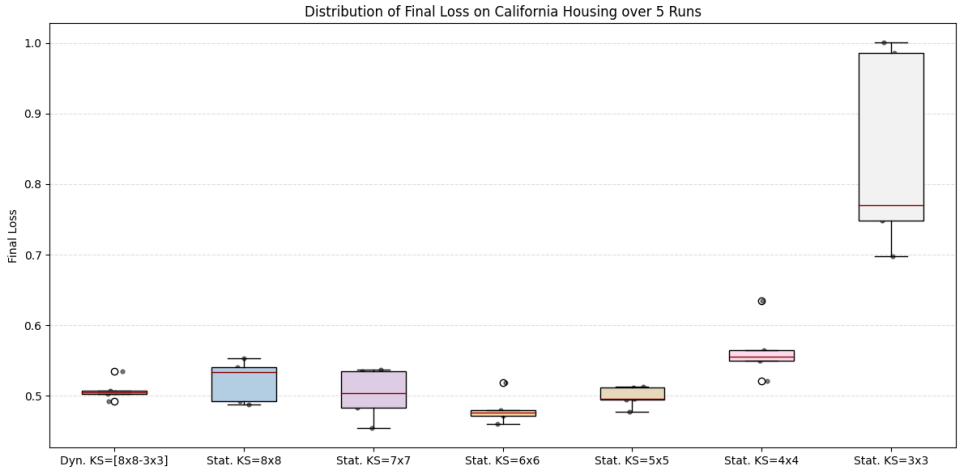

Figure 8.8: Distribution of Final Loss over 5 runs for the Big Net.

The boxplot clearly shows the catastrophic variance of the *Static 3x3* configuration. In contrast, the *Dynamic* method remains extremely stable. Although the *Static 6x6* achieved the absolute lowest median loss in this specific test, the Dynamic approach provides a nearly identical performance level without any manual tuning, acting as a reliable "safety net" against suboptimal kernel choices.

The broader predictive performance across regression metrics is presented in Figure 8.9.

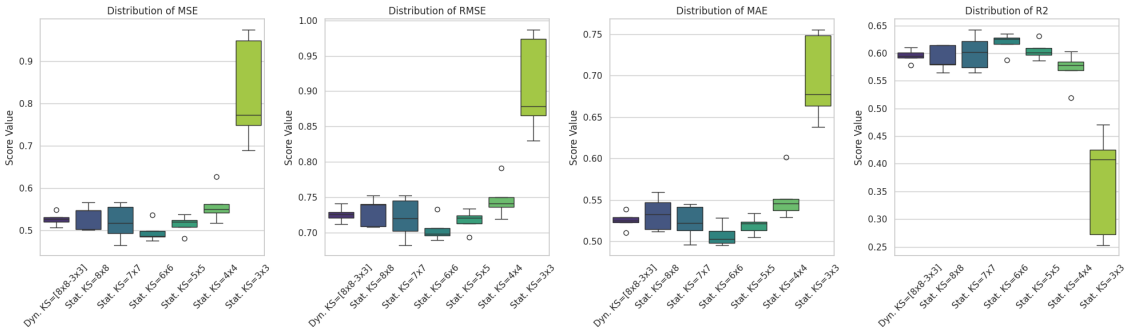

Figure 8.9: Boxplots for MSE, RMSE, MAE, and R^2 across all Big Net configurations.

These distributions confirm the general trend: the Dynamic approach matches the predictive power of the best static configurations (6x6 and 7x7). As seen in the R^2 plot, the dynamic method centers its performance around 0.60, effectively outperforming the 8x8, 4x4, and 3x3 static setups.

Summary Statistics

Table 8.4 summarizes the average metrics for the Big Net. The results prove that dynamic reshaping is a viable strategy for large networks, offering a balanced trade-off between execution time and predictive accuracy.

Metric (Avg)	**Dyn. [8x-3x]**	**Stat. 8x8**	**Stat. 7x7**	**Stat. 6x6**	**Stat. 5x5**	**Stat. 4x4**	**Stat. 3x3**
Final Loss	0.5083	0.5214	0.5025	0.4813	0.4985	0.5652	0.8406
Time (s)	704.31	708.07	716.08	712.44	702.59	698.35	696.42
MSE	0.5274	0.5335	0.5201	0.4972	0.5149	0.5600	0.8263
RMSE	0.7262	0.7302	0.7207	0.7050	0.7174	0.7479	0.9069
MAE	0.5252	0.5333	0.5236	0.5075	0.5196	0.5530	0.6964
R^2	0.5954	0.5907	0.6010	0.6185	0.6050	0.5705	0.3660

Table 8.4: Average performance summary for the Big Net architecture.

8.2.4 Final Considerations on Dynamic Kernel Reshaping

The comparative analysis across Small, Medium, and Big architectures allows for a comprehensive evaluation of the Dynamic Kernel Reshaping approach. The experimental results lead to several key conclusions regarding its performance, stability, and practical utility.

Performance and Adaptability

The primary objective was to determine if a dynamic approach could match or exceed the performance of statically defined kernels. The data suggests a nuanced but positive outcome:

- **Near-Optimal Performance:** In the Small Net, the dynamic approach achieved results nearly identical to the best static configuration (2×2), effectively closing the gap after a brief adaptation period.
- **Architectural Superiority:** In the Medium Net, the dynamic method demonstrated its maximum potential by *outperforming all static configurations*. This suggests that as network complexity increases, the ability to adapt the receptive field during training becomes a significant competitive advantage.
- **Robustness in Large Scales:** For the Big Net, while the numerical results were comparable to the best-performing static kernels (6×6), the dynamic approach proved essential in avoiding the catastrophic convergence failures observed with smaller static kernels (3×3), which were unable to handle the increased depth of the architecture.

Operational Efficiency and Parameter Tuning

As hypothesized, the most significant practical benefit of Dynamic Kernel Reshaping is the elimination of exhaustive hyperparameter searches. Selecting the optimal kernel size is typically a trial-and-error process that varies significantly with network depth and task complexity.

The dynamic schema acts as an automated optimizer: by starting with a larger receptive field to favor global feature exploration and gradually shrinking to smaller dimensions for fine-tuning, the model autonomously navigates toward a high-performance state.

8.3 Dynamic Quantization Factor Results

This section investigates the impact of dynamically adjusting the quantization factor during the training process. The primary hypothesis is that increasing the quantization resolution when the loss improvement stalls can allow the A^* algorithm to perform finer weight adjustments, potentially escaping local minima or refining the solution quality.

The dynamic approach is compared against several static quantization factors ($QF \in \{10^1, 10^2, 10^3, 10^4\}$) to evaluate whether an adaptive strategy can achieve the high precision of a large QF without the computational search burden or the convergence difficulties often associated with high-resolution discrete spaces.

8.3.1 Small Net Results

For the Small Net architecture, the following parameters were utilized to establish the baseline and dynamic behavior:

- **Weight Kernel Size:** $[2, 2]$ (Static).
- **Bias Kernel Size:** [2] (Static).
- **Strides:** 2.
- **Dynamic QF Range:** Min 10, Max $10,000$ (Multiplier: 10).
- **Runs:** 5 for each configuration.

Visual Analysis of Training and Stability

The convergence behavior, as depicted in Figure 8.10, reveals a stark contrast between low and high quantization factors. Low static factors (10^1 and 10^2) lead to rapid initial convergence. In contrast, static factors of 10^3 and 10^4 struggle significantly, failing to achieve a competitive loss within the same iteration limit.

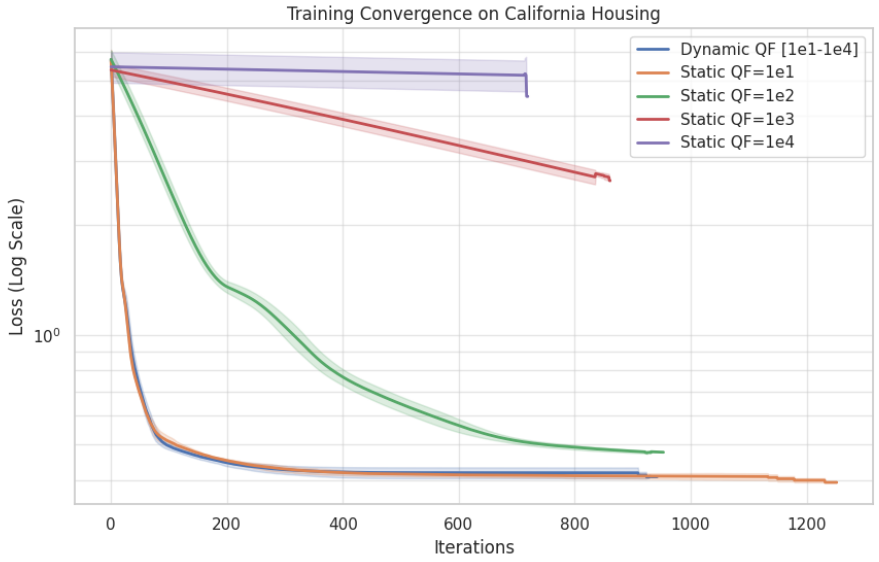

Figure 8.10: Training convergence comparison for various Quantization Factors (Log Scale).

The *Dynamic QF* strategy (blue line) successfully mirrors the rapid descent of the $QF = 10$ configuration. This suggests that starting with a lower resolution allows for broad, efficient exploration of the weight space, while the dynamic increases (triggered by stalls) ensure the model maintains progress.

Figure 8.11 illustrates the distribution of the final loss. The Dynamic QF approach demonstrates high stability, with a median loss of approximately 0.41, nearly identical to the best-performing static model ($QF = 10$).

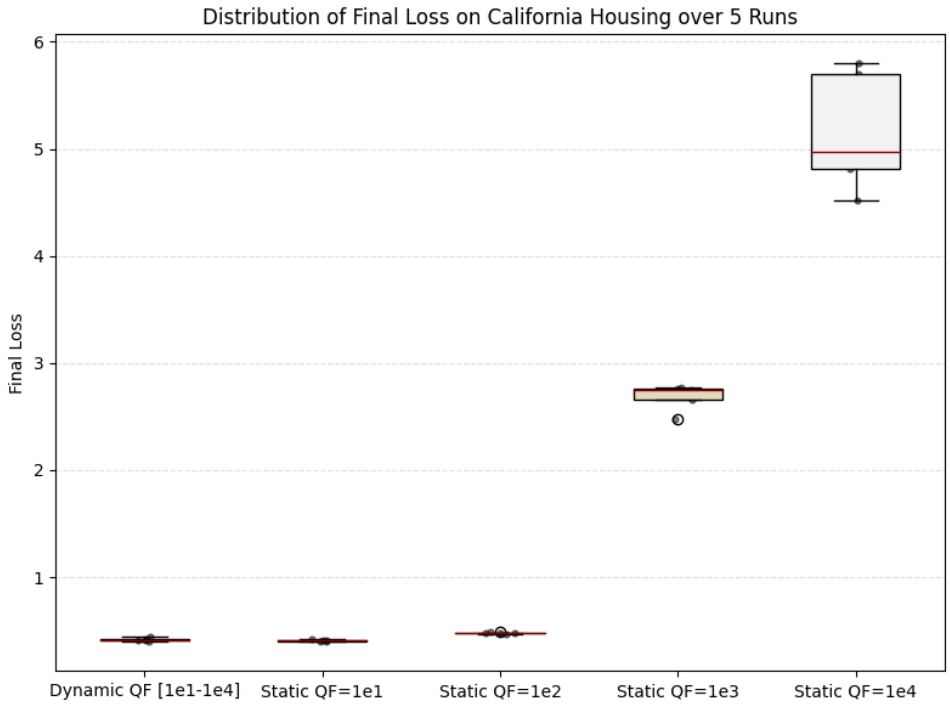

Figure 8.11: Distribution of Final Loss over 5 runs for Quantization Factor tests.

As the static quantization factor increases beyond 10^2, we observe not only a higher average loss but also a significant increase in variance. This confirms that excessively high quantization at the start of training makes the search landscape too complex for the algorithm to navigate effectively.

The regression metrics across the 5 runs are summarized in Figure 8.12.

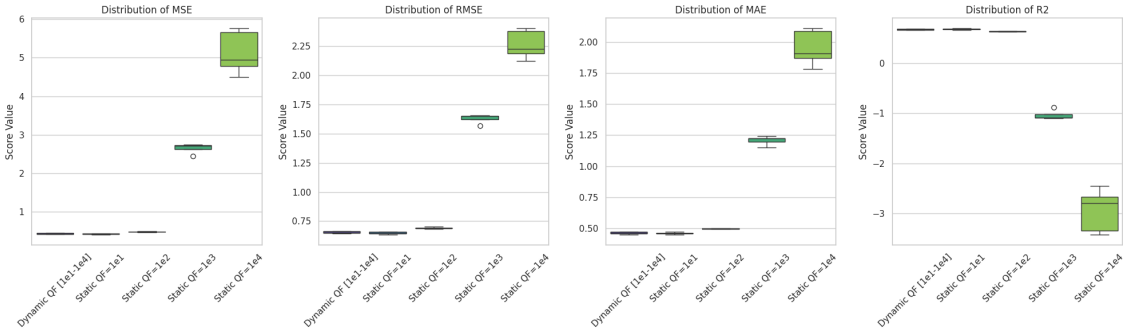

Figure 8.12: Regression metrics distribution (MSE, RMSE, MAE, and R^2) for Quantization Factor configurations.

The metrics highlight that the *Dynamic QF* and *Static QF=10, 100* are the only configurations achieving a positive and high R^2 score (approximately 0.67). For static factors of 10^3 and 10^4, the R^2 values become negative, indicating that the models perform worse than a horizontal baseline, further justifying the need for a dynamic or low-starting quantization approach.

Summary Statistics

The average values for all metrics, compiled in Table 8.5, provide a definitive comparison of the performance levels.

Metric (Avg)	Dynamic [1e1-1e4]	Static 1e1	Static 1e2	Static 1e3	Static 1e4
Final Loss	0.4184	0.4085	0.4770	2.6815	5.1592
Time (s)	667.96	777.83	644.30	579.98	492.87
MSE	0.4268	0.4208	0.4788	2.6506	5.1290
RMSE	0.6532	0.6486	0.6919	1.6277	2.2621
MAE	0.4634	0.4618	0.4983	1.2077	1.9496
R^2	0.6725	0.6772	0.6327	-1.0336	-2.9352

Table 8.5: Average performance summary for Small Net (Quantization Factor Study).

8.3.2 Medium Net Results

The evaluation of the Dynamic Quantization Factor (QF) was extended to the Medium Net architecture to assess how increased model complexity interacts with

search space resolution. The configuration remained consistent with the previous test to ensure comparability:

- **Weight Kernel Size:** $[2, 2]$ (Static).
- **Bias Kernel Size:** [2] (Static).
- **Strides:** 2.
- **Dynamic QF Range:** Min 10, Max $10,000$ (Multiplier: 10).
- **Runs:** 5 for each configuration.

Visual Analysis of Training and Stability

As shown in Figure 8.13, the convergence trends for the Medium Net reinforce the observations from the Small Net but with more pronounced performance gaps. Static factors of 10^3 and 10^4 show almost no effective optimization, maintaining a high loss throughout the iterations.

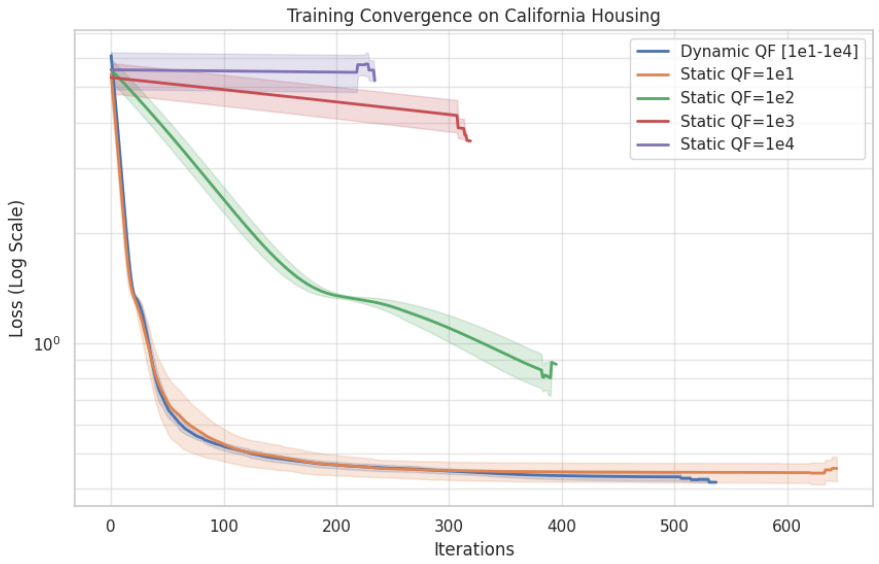

Figure 8.13: Training convergence comparison for Medium Net (Log Scale).

Interestingly, the *Dynamic QF* strategy (blue line) demonstrates a highly efficient trajectory, converging rapidly and reaching a final loss state that is slightly lower

than the $QF = 10$ static baseline. This suggests that as the network grows in size, the ability to dynamically refine weight updates becomes increasingly valuable for fine-tuning.

Figure 8.14 highlights the stability of these results. The Dynamic QF approach maintains a very low variance (Std: 0.0108), outperforming the Static $QF = 10$ in terms of consistency.

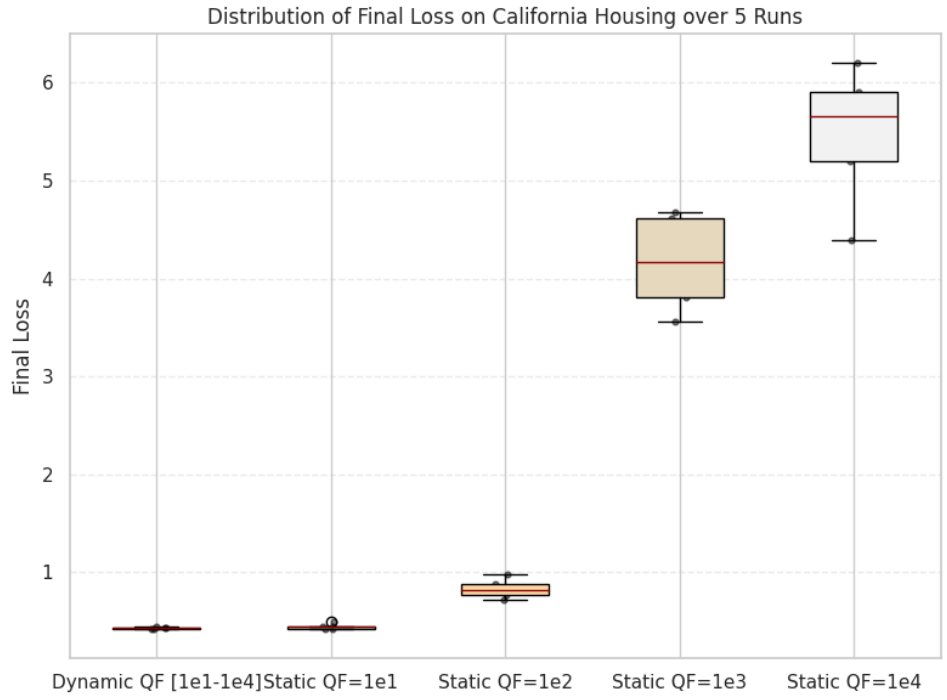

Figure 8.14: Distribution of Final Loss for Medium Net configurations.

A critical observation is the behavior of the Static $QF = 10^2$ configuration. While it was somewhat competitive in the Small Net, its performance degrades significantly in the Medium Net (Avg Loss 0.83 vs 0.47 in Small Net). This indicates that higher-dimensional networks are more sensitive to initial discretization errors.

The regression metrics for the Medium Net are visualized in Figure 8.15.

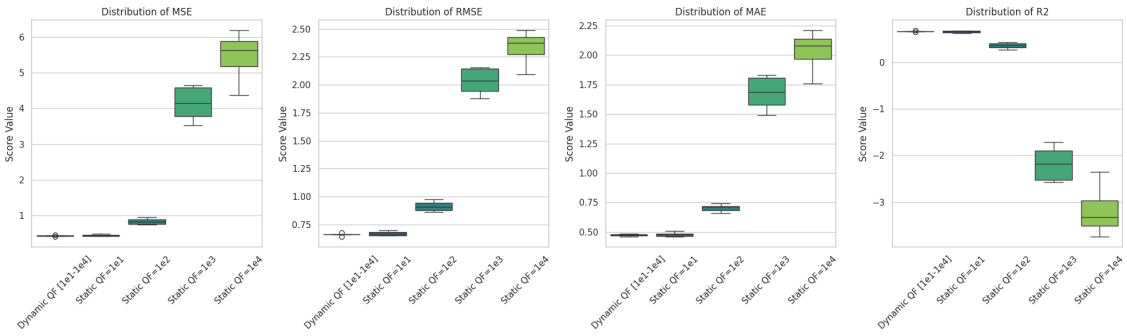

Figure 8.15: Regression metrics distribution for Medium Net configurations.

The metrics confirm that the *Dynamic QF* achieves the best overall performance, with an average R^2 of 0.6646, slightly surpassing the 0.6535 achieved by the Static $QF = 10$. Configurations with $QF \geq 10^3$ fail to produce meaningful predictive models, as evidenced by the high MSE and negative R^2 values.

Summary Statistics

Table 8.6 provides the numerical breakdown of the average metrics for the Medium Net.

Metric (Avg)	Dynamic [1e1-1e4]	Static 1e1	Static 1e2	Static 1e3	Static 1e4
Final Loss	0.4308	0.4423	0.8327	4.1656	5.4733
Time (s)	618.54	739.54	465.80	379.06	283.61
MSE	0.4371	0.4516	0.8373	4.1377	5.4457
RMSE	0.6611	0.6717	0.9140	2.0312	2.3295
MAE	0.4742	0.4794	0.7030	1.6773	2.0293
R^2	0.6646	0.6535	0.3576	-2.1746	-3.1782

Table 8.6: Average performance summary for Medium Net (Quantization Factor Study).

8.3.3 Big Net Results

Finally, the study concludes with the Big Net architecture. As the number of parameters increases, the search space becomes exponentially more complex, making the initial selection of a quantization factor a determining factor for training success. The experimental parameters remained as follows:

- **Weight Kernel Size:** $[2, 2]$ (Static).
- **Bias Kernel Size:** $[2]$ (Static).
- **Strides:** 2.
- **Dynamic QF Range:** Min 10, Max $10,000$ (Multiplier: 10).
- **Runs:** 5 for each configuration.

Visual Analysis of Training and Stability

The convergence patterns for the Big Net, shown in Figure 8.16, demonstrate that the A^* algorithm's efficiency is highly dependent on low-resolution starts for large architectures. Similar to previous networks, $QF = 10^3$ and $QF = 10^4$ are unable to achieve meaningful optimization, plateauing almost immediately.

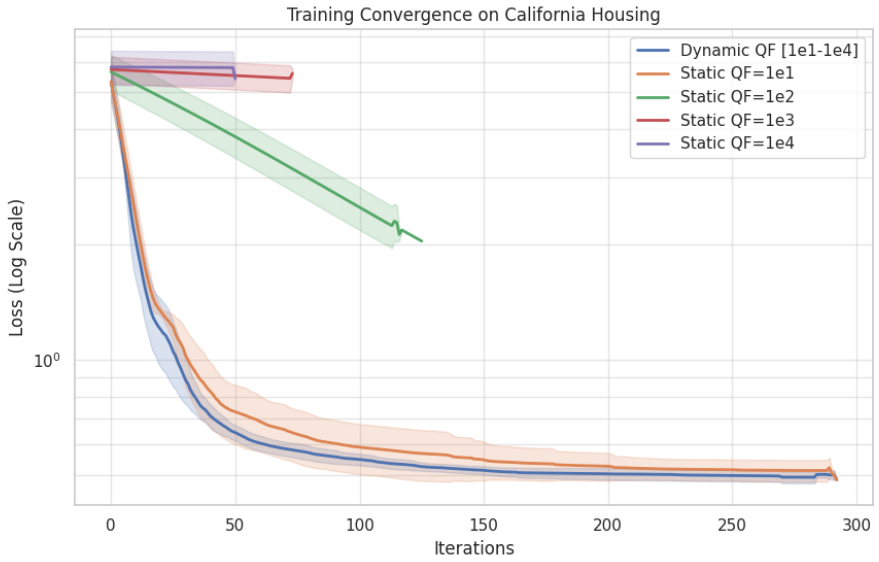

Figure 8.16: Training convergence comparison for Big Net (Log Scale).

The *Dynamic QF* strategy (blue line) exhibits a superior convergence rate even compared to the best static configuration ($QF = 10$). In this high-dimensional setting, the dynamic approach proves to be the most robust, navigating the complex loss landscape more effectively than any fixed-resolution method.

Figure 8.17 confirms this superiority. Not only does the Dynamic QF achieve the lowest average loss, but it also demonstrates significantly lower variance compared to the $QF = 10$ static run, which shows several higher-loss outliers.

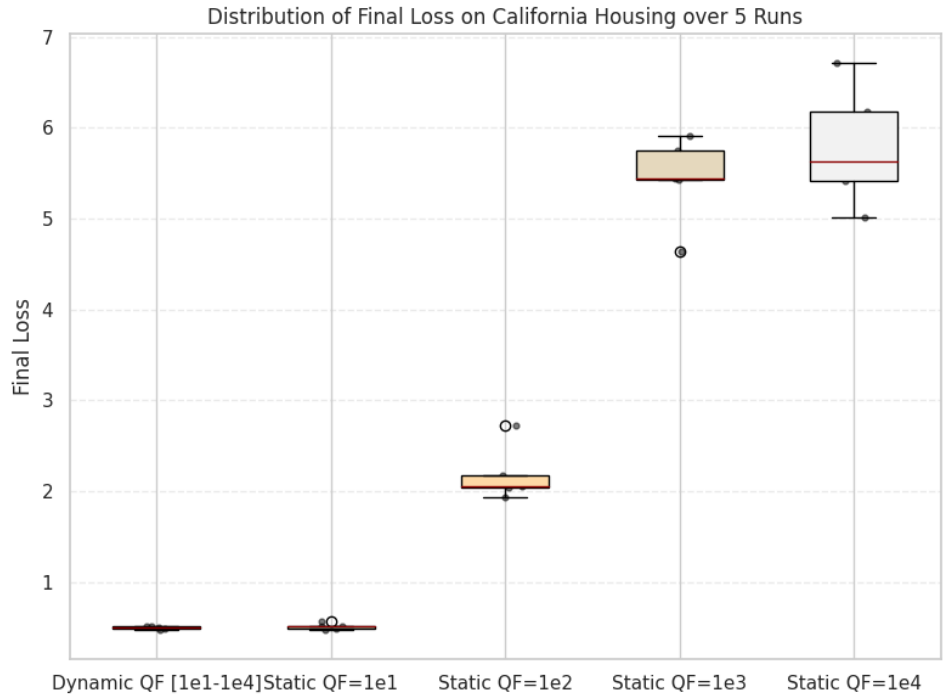

Figure 8.17: Distribution of Final Loss for Big Net configurations.

The failure of the $QF = 10^2$ configuration is even more evident here than in the Medium Net; while it manages a slow descent, its final loss (Avg 2.18) remains far from the optimal region, highlighting that higher parameter counts require even more careful discretization management.

The comparative regression metrics are presented in Figure 8.18.

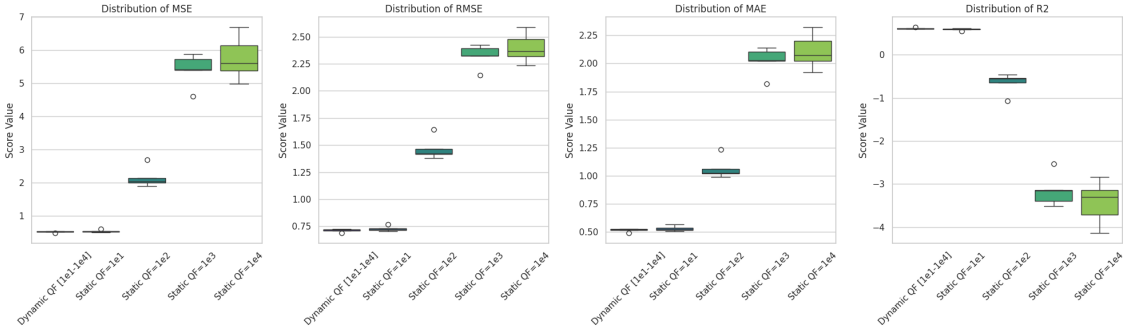

Figure 8.18: Regression metrics distribution for Big Net configurations.

The *Dynamic QF* achieves an average R^2 of 0.6106, outperforming the 0.5931 of the static $QF = 10$ case. The significant drop in R^2 for $QF = 10^2$ (which becomes negative at -0.65) underscores the "discretization trap" where a slightly too-high resolution prevents the algorithm from finding the global descent direction in large models.

Summary Statistics

The numerical results for the Big Net are detailed in Table 8.7.

Metric (Avg)	Dynamic [1e1-1e4]	Static 1e1	Static 1e2	Static 1e3	Static 1e4
Final Loss	0.4959	0.5133	2.1827	5.4331	5.7914
Time (s)	703.85	716.39	300.36	190.66	135.31
MSE	0.5075	0.5303	2.1525	5.4062	5.7643
RMSE	0.7123	0.7279	1.4642	2.3231	2.3977
MAE	0.5146	0.5290	1.0635	2.0227	2.1074
R^2	0.6106	0.5931	-0.6515	-3.1478	-3.4226

Table 8.7: Average performance summary for Big Net (Quantization Factor Study).

8.3.4 Conclusion on Dynamic Quantization Factor Study

The experimental campaign across three architectures of increasing complexity — Small, Medium, and Big Net — provides a clear understanding of how the quantization factor (QF) influences the A^* search space in neural network training.

- **Robustness against the "Discretization Trap":** The most significant finding is that while low static quantization ($QF = 10$) performs well, static factors

from 10^2 upwards lead to a rapid degradation of performance as the network size increases. The *Dynamic QF* strategy effectively mitigates this risk by starting with a coarse resolution to find the global descent direction and only increasing precision when local stalls occur.

- **Performance Trends in Scalability:** For Medium and Big Net architectures, the *Dynamic QF* approach did not only match but consistently **outperformed** the best static configurations in terms of final MSE and R^2 score. While the margin of improvement in the Small Net was negligible, the benefits became more pronounced as the parameter space expanded, suggesting that adaptive precision is a key requirement for scaling A^*-based training.

- **Operational Efficiency:** Beyond the raw metrics, the primary advantage of the dynamic approach is the **elimination of the hyperparameter search process**. Manually identifying the optimal QF for a specific architecture is a computationally expensive trial-and-error task. The dynamic strategy offers a "set-and-forget" solution that guarantees near-optimal or superior performance across different network depths.

- **Convergence Stability:** As evidenced by the boxplots, the Dynamic QF approach significantly reduced the variance of the final loss compared to static methods. This stability is crucial for ensuring reliable training outcomes across multiple runs, regardless of weight initialization.

In conclusion, the **Dynamic Quantization Factor** is an essential optimization. It provides the algorithm with the flexibility to balance broad exploration with high-precision refinement, making the training process both more accurate and significantly more user-friendly by removing a critical hyperparameter from manual tuning.

8.4 Random Neighbors Generation: Grid Search on Sampling Parameters

This section explores the *Random Neighbors Generation* strategy, which introduces a stochastic approach to weight optimization. Unlike the deterministic kernel-based methods, this scheme relies on two primary hyperparameters to define the search neighborhood: the *Perturbation Ratio* and the *Search Coverage Ratio*.

The *Perturbation Ratio* determines the cardinality of the subset of parameters updated simultaneously within each layer. It is expressed as a percentage of the total number of parameters per layer ($|W_{layer}| \times$ Perturbation Ratio). The *Search Coverage Ratio* governs the total number of child nodes generated for each search step, similarly defined as a percentage of the total parameters in the layer.

The objective of this grid search is to identify the optimal balance between the intensity of each update (perturbation) and the breadth of the local search (coverage) to maximize convergence efficiency in a discrete state-space.

8.4.1 Small Net Results

For the Small Net architecture, a grid search was conducted using the following parameter combinations:

- **Perturbation Ratios:** $[0.01, 0.05, 0.1]$ $(1\%, 5\%, 10\%)$.
- **Search Coverage Ratios:** $[0.05, 0.1, 0.2]$ $(5\%, 10\%, 20\%)$.
- **Iterations:** 2000.
- **Runs:** 5 for each unique pair.

Visual Analysis of Convergence and Loss Distribution

The training trajectories for all nine configurations are illustrated in Figure 8.19. The results indicate a clear sensitivity to the perturbation ratio.

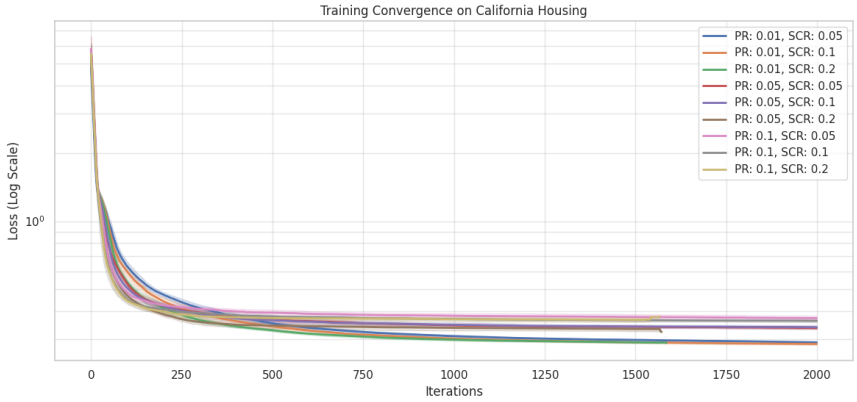

Figure 8.19: Training convergence for Small Net using various Random Sampling configurations (Log Scale).

The configurations with a lower perturbation ratio (1%, depicted in blue and green hues) exhibit the most stable and deepest descent. As the perturbation ratio increases to 5% and 10%, the algorithm struggles to refine the solution, likely due to "jumping" over narrow local minima.

The stability of these stochastic updates is further examined in the final loss distribution shown in Figure 8.20.

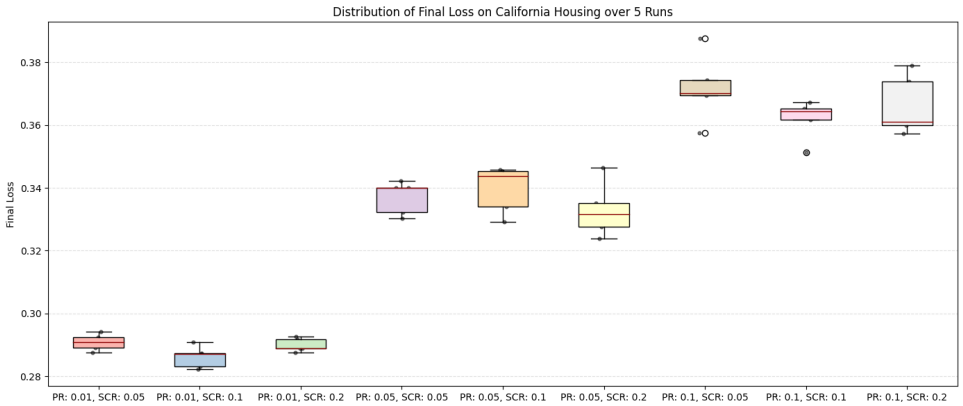

Figure 8.20: Distribution of Final Loss over 5 runs across the grid search space.

The best performing configuration is **Perturbation Ratio: 0.01** with a **Search Ratio: 0.1**, which achieves the lowest average loss with remarkably low variance. Interestingly, increasing coverage (Search Ratio) from 0.1 to 0.2 does not significantly improve the final loss for the 0.01 perturbation case, but it substantially increases the computational time.

The regression metrics across the grid are summarized in Figure 8.21.

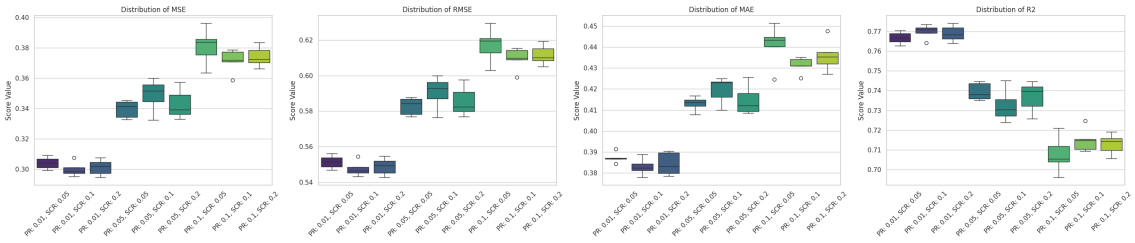

Figure 8.21: Distribution of MSE, RMSE, MAE, and R^2 for the Random Sampling Grid Search.

The R^2 scores confirm that smaller perturbations are vastly superior for this task, with configurations at 1% perturbation reaching R^2 values near 0.77. This suggests that for a network of this size, high-precision, small-scale stochastic adjustments are more effective than larger, more aggressive updates.

Summary Statistics

Tables below provide the numerical breakdown of the average performance for each tested configuration.

Pert. / Search	**0.01 / 0.05**	**0.01 / 0.1**	**0.01 / 0.2**	**0.05 / 0.05**	**0.05 / 0.1**
Avg Final Loss	0.2909	0.2862	0.2900	0.3369	0.3396
Avg Time (s)	325.44	562.40	855.63	278.16	535.81
Avg MSE	0.3040	0.2998	0.3012	0.3396	0.3488
Avg RMSE	0.5514	0.5476	0.5488	0.5827	0.5905
Avg MAE	0.3872	0.3829	0.3843	0.4130	0.4195
Avg R^2	0.7668	0.7700	0.7689	0.7395	0.7324

Table 8.8: Average statistical metrics for Small Net Random Sampling (Part 1).

Pert. / Search	0.05 / 0.2	0.1 / 0.05	0.1 / 0.1	0.1 / 0.2
Avg Final Loss	0.3329	0.3718	0.3620	0.3662
Avg Time (s)	823.20	270.92	524.14	805.04
Avg MSE	0.3429	0.3810	0.3715	0.3741
Avg RMSE	0.5855	0.6172	0.6095	0.6116
Avg MAE	0.4148	0.4408	0.4313	0.4359
Avg R^2	0.7369	0.7077	0.7150	0.7130

Table 8.9: Average statistical metrics for Small Net Random Sampling (Part 2).

8.4.2 Medium Net Results

For the Medium Net architecture, the grid search was conducted using the same parameter combinations to assess how increased network complexity influences sampling dynamics:

- **Perturbation Ratios:** $[0.01, 0.05, 0.1]$ $(1\%, 5\%, 10\%)$.
- **Search Coverage Ratios:** $[0.05, 0.1, 0.2]$ $(5\%, 10\%, 20\%)$.
- **Iterations:** 2000.
- **Runs:** 5 for each unique pair.

Visual Analysis of Convergence and Loss Distribution

The training trajectories for the nine configurations on the Medium Net are illustrated in Figure 8.22.

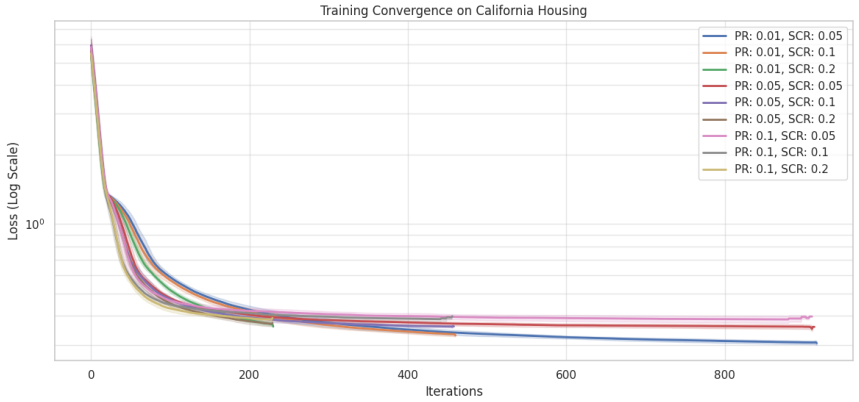

Figure 8.22: Training convergence for Medium Net using various Random Sampling configurations (Log Scale).

In this architecture, sensitivity to the *perturbation ratio* remains the dominant factor. It is observed that the configuration with 1% perturbation and 5% coverage (Pert. Ratio: 0.01, Search Ratio: 0.05) produces the most rapid and deepest descent. Compared to the Small Net, the increased number of parameters seems to make a very narrow local search (low Search Ratio) more effective, indicating that in higher-dimensional spaces, over-exploration through too many random directions can be less efficient than precision in individual steps.

The stability of these stochastic updates is further examined in the final loss distribution shown in Figure 8.23.

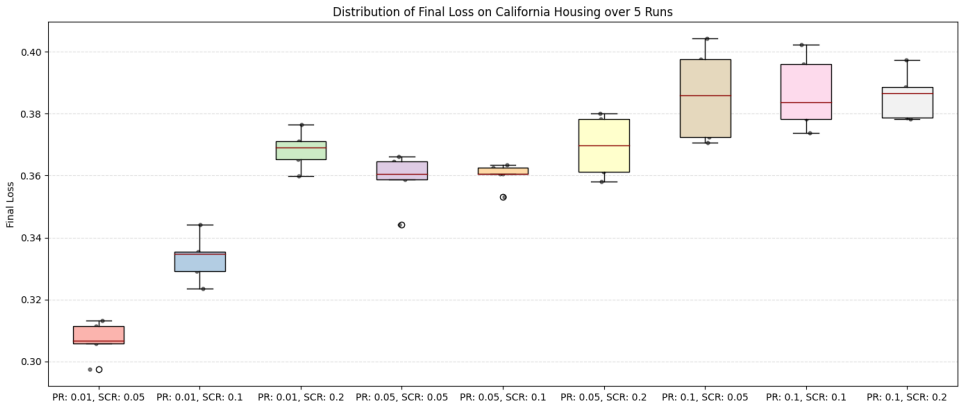

Figure 8.23: Distribution of Final Loss over 5 runs across the grid search space for Medium Net.

The **Perturbation Ratio: 0.01** with **Search Ratio: 0.05** configuration is confirmed as the best performer, achieving the lowest average loss (0.3068) with extremely low variance (0.000030). Interestingly, for this network, increasing the coverage (Search Ratio) to 0.1 or 0.2 actually degrades the final loss, suggesting a risk of over-exploration that leads the algorithm away from the most promising local minima.

The regression metrics for the Medium Net are summarized in Figure 8.24.

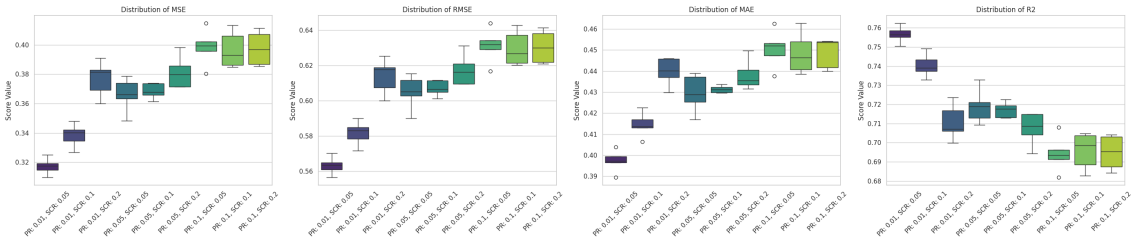

Figure 8.24: Distribution of MSE, RMSE, MAE, and R^2 for the Medium Net Random Sampling Grid Search.

The R^2 scores highlight that predictive performance decreases linearly as the perturbation increases. The optimal configuration reaches an average R^2 of approximately 0.7566, demonstrating that even with a deeper network, random sampling with small increments is capable of effectively modeling the California Housing dataset.

Summary Statistics

The tables below provide the numerical breakdown of the average performance for each tested configuration on the Medium Net architecture.

Pert. / Search	**0.01 / 0.05**	**0.01 / 0.1**	**0.01 / 0.2**	**0.05 / 0.05**	**0.05 / 0.1**
Avg Final Loss	0.3068	0.3334	0.3684	0.3588	0.3601
Avg Time (s)	802.43	790.59	781.27	788.71	783.09
Avg MSE	0.3173	0.3384	0.3770	0.3662	0.3686
Avg RMSE	0.5632	0.5817	0.6139	0.6051	0.6071
Avg MAE	0.3973	0.4145	0.4397	0.4295	0.4313
Avg R^2	0.7566	0.7404	0.7108	0.7191	0.7172

Table 8.10: Average statistical metrics for Medium Net Random Sampling (Part 1).

Pert. / Search	0.05 / 0.2	0.1 / 0.05	0.1 / 0.1	0.1 / 0.2
Avg Final Loss	0.3695	0.3862	0.3868	0.3859
Avg Time (s)	777.98	774.91	775.43	779.09
Avg MSE	0.3814	0.3985	0.3967	0.3976
Avg RMSE	0.6175	0.6312	0.6297	0.6305
Avg MAE	0.4382	0.4505	0.4485	0.4486
Avg R^2	0.7074	0.6942	0.6957	0.6949

Table 8.11: Average statistical metrics for Medium Net Random Sampling (Part 2).

8.4.3 Big Net Results

For the Big Net architecture, the grid search explores how stochastic optimization scales with a significantly larger parameter space. The test conditions remain consistent with the previous architectures:

- **Perturbation Ratios:** $[0.01, 0.05, 0.1]$ $(1\%, 5\%, 10\%)$.
- **Search Coverage Ratios:** $[0.05, 0.1, 0.2]$ $(5\%, 10\%, 20\%)$.
- **Iterations:** 2000.
- **Runs:** 5 for each unique pair.

Visual Analysis of Convergence and Loss Distribution

The training trajectories for the Big Net are presented in Figure 8.25.

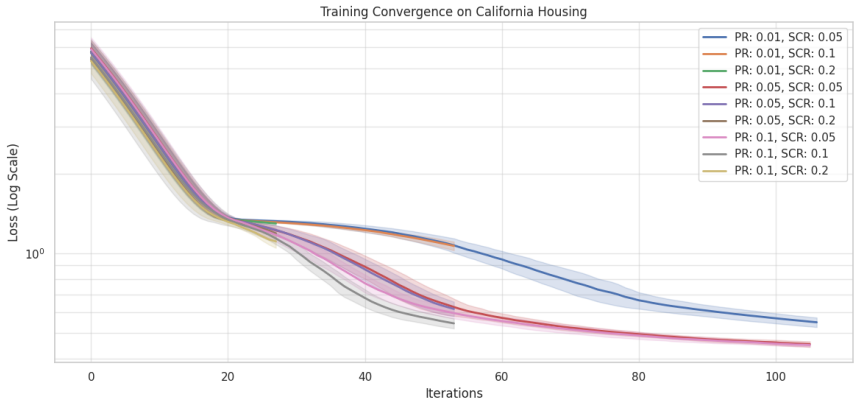

Figure 8.25: Training convergence for Big Net using various Random Sampling configurations (Log Scale).

A significant shift in behavior is observed compared to the smaller networks. While in the Small Net a low perturbation was universally better, the Big Net shows that a higher *Perturbation Ratio* (0.05 or 0.1), when paired with a very low *Search Coverage Ratio* (0.05), results in the most effective optimization. This suggests that in high-dimensional spaces, making more substantial updates to a smaller number of candidate neighbors is more effective than fine-grained updates across a wide search breadth.

The distribution of final loss values is shown in Figure 8.26, highlighting a clear "curse of dimensionality" effect.

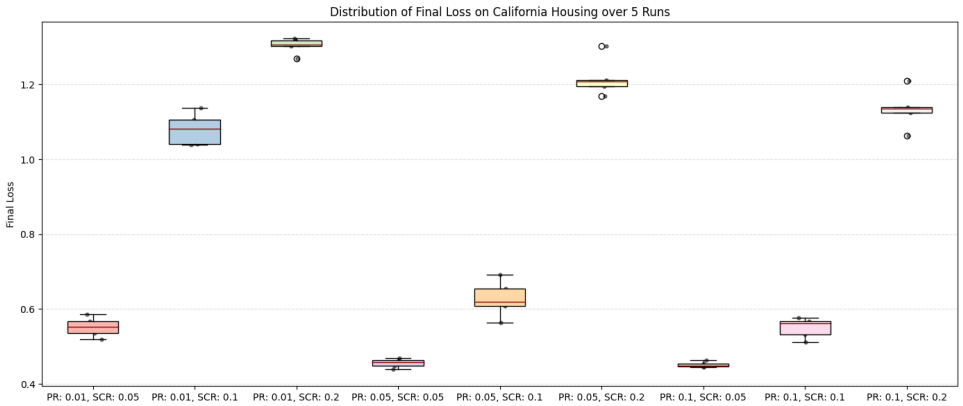

Figure 8.26: Distribution of Final Loss over 5 runs across the grid search space for Big Net.

The best-performing configurations are **Perturbation Ratio: 0.1 / Search Ratio: 0.05** (Avg Loss: 0.4518) and **Perturbation Ratio: 0.05 / Search Ratio: 0.05** (Avg Loss: 0.4554). Conversely, any configuration with a high Search Ratio (0.2) performs poorly, with losses exceeding 1.1, indicating that searching too many random directions in such a large space leads to a lack of refinement.

The regression metrics for the Big Net are visualized in Figure 8.27.

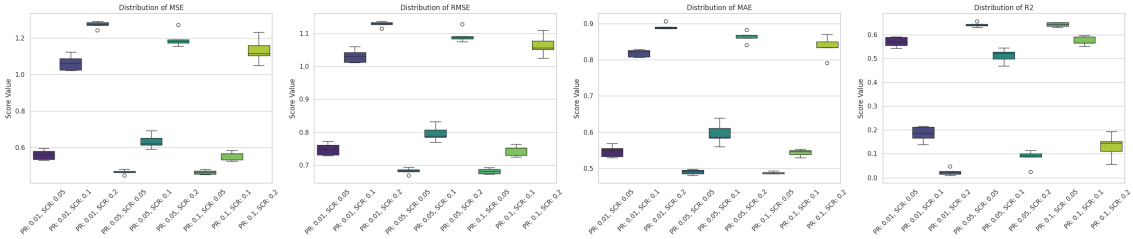

Figure 8.27: Distribution of MSE, RMSE, MAE, and R^2 for the Big Net Random Sampling Grid Search.

The R^2 scores confirm that the predictive power of the model is highly sensitive to the search ratio in large networks. The optimal configurations reach an R^2 of approximately 0.6428. However, when the Search Ratio increases to 0.2, the R^2 drops dramatically toward zero (0.02 to 0.13), showing that the algorithm fails to find a meaningful mapping when the search is too broad and stochastic.

Summary Statistics

The tables below summarize the average statistical performance for all tested configurations on the Big Net.

Pert. / Search	**0.01 / 0.05**	**0.01 / 0.1**	**0.01 / 0.2**	**0.05 / 0.05**	**0.05 / 0.1**
Avg Final Loss	0.5523	1.0809	1.3036	0.4554	0.6274
Avg Time (s)	733.17	726.02	739.29	727.94	729.05
Avg MSE	0.5609	1.0637	1.2724	0.4671	0.6351
Avg RMSE	0.7488	1.0312	1.1280	0.6834	0.7967
Avg MAE	0.5471	0.8186	0.8918	0.4898	0.5959
Avg R^2	0.5696	0.1839	0.0238	0.6416	0.5127

Table 8.12: Average statistical metrics for Big Net Random Sampling (Part 1).

Pert. / Search	0.05 / 0.2	0.1 / 0.05	0.1 / 0.1	0.1 / 0.2
Avg Final Loss	1.2176	0.4518	0.5501	1.1347
Avg Time (s)	739.12	727.02	730.05	735.02
Avg MSE	1.1942	0.4656	0.5557	1.1316
Avg RMSE	1.0927	0.6823	0.7453	1.0634
Avg MAE	0.8643	0.4882	0.5433	0.8360
Avg R^2	0.0837	0.6428	0.5737	0.1318

Table 8.13: Average statistical metrics for Big Net Random Sampling (Part 2).

8.4.4 Conclusion on Random Neighbors Generation Study

The grid search analysis conducted across Small, Medium, and Big Net architectures provides several key insights into the behavior of stochastic training within a discrete state-space search.

- **Architecture-Dependent Sensitivity:** The study demonstrates that the optimal hyperparameter configuration is strictly linked to the network's dimensionality. For the **Small Net**, a fine-grained approach (1% perturbation with 10% coverage) yielded the best results. However, as the parameter space expanded in the **Big Net**, the algorithm required more aggressive updates (5-10% perturbation) but a much narrower search breadth (5% coverage) to avoid the "noise" of excessive random exploration.

- **The Optimal Hyperparameter Window:** Despite architectural variances, the most effective configurations consistently fell within a specific range: **Perturbation Ratios** between $[0.01, 0.1]$ and **Search Coverage Ratios** between $[0.05, 0.1]$. Deviating outside these bounds — particularly by increasing search coverage to 20% — consistently resulted in diminishing returns or performance degradation due to over-exploration.

- **Stochastic vs. Deterministic Search:** Most significantly, the random sampling approach consistently outperformed the kernel-based methods tested in previous sections. While kernels provide a structured update, random sampling allows the A^* algorithm to explore a more diverse set of directions in the weight landscape. This stochasticity appears to be a **primary driver for performance**, enabling the model to achieve higher R^2 scores and lower MSE values across all tested architectures.

- **Computational Efficiency:** Random sampling proved to be computationally robust. Even with lower search coverage, the ability to find high-quality descent

directions allowed for faster convergence compared to static high-resolution quantization methods, making it a highly scalable alternative for complex neural network training.

In conclusion, **Random Neighbors Generation** transforms the training process into a more effective stochastic optimization task. By balancing the intensity of updates with a controlled search breadth, this method provides a superior ability to navigate the complex loss surfaces of deep learning models compared to rigid, kernel-driven architectures.

8.5 Neighbors Generation Strategies Comparison

This section evaluates and compares the three proposed neighbor generation strategies—*Single Kernel*, *Layer-wise Kernels*, and *Random Neighbors Generation*—using the optimal hyperparameters identified in the previous tuning phases. To isolate the intrinsic performance of each generation logic, the Beam Search optimization is omitted in these tests.

The goal is to determine which strategy provides the most effective exploration of the weight space and achieves the lowest convergence error on the California Housing benchmark.

8.5.1 Small Net Results

The Small Net architecture was tested using the following optimized configurations:

- **Single Kernel:** Weight kernel $[2, 2]$, Bias kernel $[2]$, Stride 2.
- **Layer-wise Kernels:**
  - Weight kernels: $[[2, 2], [2, 2], [1, 2]]$
  - Bias kernels: $[[2], [2], [1]]$
  - Weight strides: $[[2, 2], [2, 2], [2, 1]]$
  - Bias strides: $[[2], [2], [1]]$
- **Random Sampling:** Perturbation ratio 0.01, Search coverage ratio 0.1.

Visual Analysis of Training and Stability

The training convergence comparison is illustrated in Figure 8.28. While all strategies exhibit a similar initial descent, a significant gap emerges after the first 250 iterations.

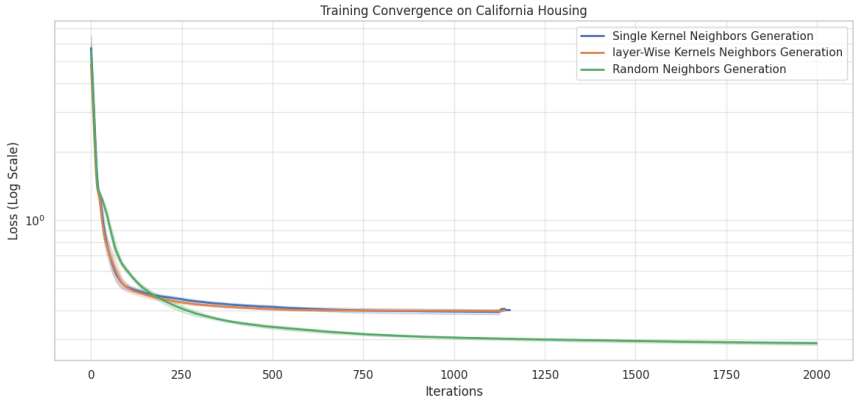

Figure 8.28: Training convergence comparison between Single Kernel, Layer-wise, and Random strategies (Log Scale).

The *Random Neighbors Generation* strategy (green line) shows a markedly superior convergence profile, continuing to reduce the loss long after the kernel-based methods have reached a plateau. This suggests that stochastic sampling allows the A^* algorithm to navigate the high-dimensional loss surface more flexibly than the fixed structural updates of the kernel methods.

The distribution of the final loss values across 5 independent runs is shown in Figure 8.29.

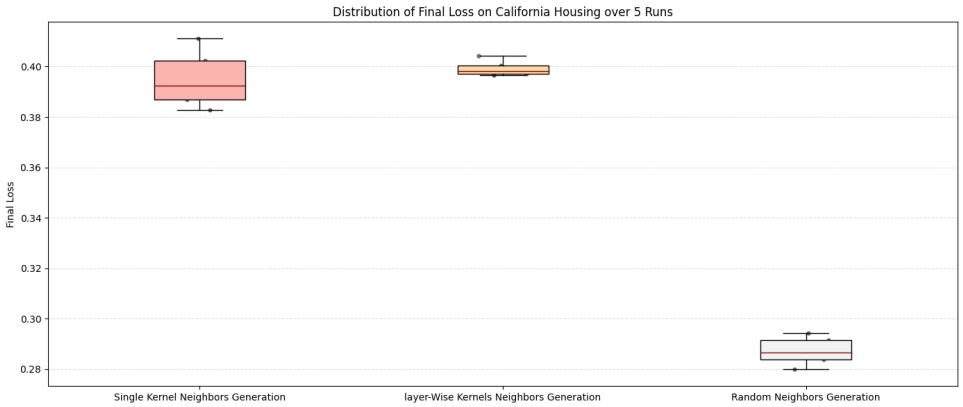

Figure 8.29: Distribution of Final Loss for the three generation strategies on Small Net.

The boxplot confirms that the Random strategy is not only more effective but also highly stable, maintaining a lower median loss (0.286) compared to the Single Kernel (0.392) and Layer-wise (0.398) approaches.

The regression metrics for the Small Net are visualized in Figure 8.30.

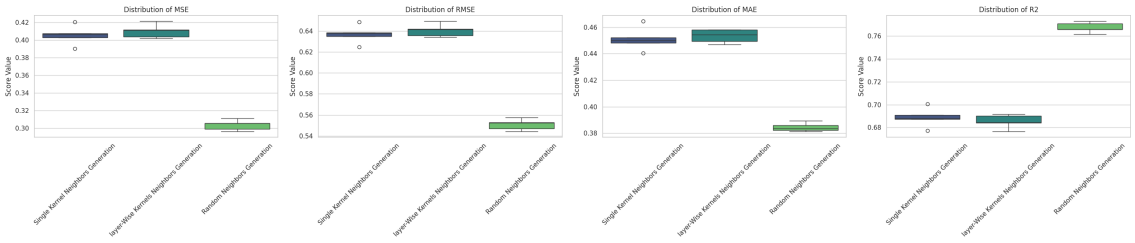

Figure 8.30: Regression metrics distribution (MSE, RMSE, MAE, R^2) across generation strategies.

The R^2 scores further validate the superiority of the Random approach, which achieves values centered around 0.77, whereas both kernel strategies cluster significantly lower, near 0.69.

Summary Statistics

The numerical performance summary in Table 8.14 highlights the clear advantage of stochastic sampling for this architecture.

Metric (Avg)	**Random (0.01 / 0.1)**	**Single Kernel**	**Layer-wise Kernels**
Avg Final Loss	0.2872	0.3951	0.3993
Avg Time (s)	535.60	777.80	753.39
Avg MSE	0.3035	0.4055	0.4101
Avg RMSE	0.5508	0.6368	0.6404
Avg MAE	0.3844	0.4511	0.4534
Avg R^2	0.7672	0.6889	0.6853

Table 8.14: Average statistical comparison of generation strategies for Small Net.

8.5.2 Medium Net Results

The Medium Net architecture was evaluated using the optimal settings derived from the previous grid search. The configurations compared are:

- **Single Kernel:** Weight kernel $[4, 4]$, Bias kernel $[4]$, Stride 4.
- **Layer-wise Kernels:**
  - Weight kernels: $[[4, 4], [4, 4], [4, 4], [1, 4]]$

  - Bias kernels: $[[4], [4], [4], [1]]$
  - Weight strides: $[[4, 4], [4, 4], [4, 4], [4, 1]]$
  - Bias strides: $[[4], [4], [4], [1]]$

- **Random Sampling:** Perturbation ratio 0.01, Search coverage ratio 0.05.

Visual Analysis of Training and Stability

The convergence dynamics for the Medium Net, shown in Figure 8.31, reveal that the gap between stochastic and deterministic strategies widens as network depth increases.

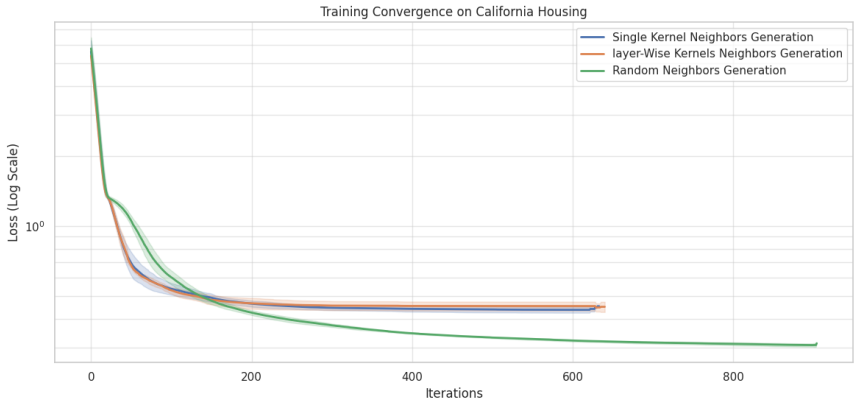

Figure 8.31: Training convergence comparison for Medium Net (Log Scale).

The *Random Neighbors Generation* (green line) continues to refine the loss effectively, whereas both the *Single Kernel* and *Layer-wise* methods reach a performance plateau relatively early, around iteration 400. This confirms that for larger architectures, the rigid geometric updates of kernels struggle to find effective descent directions in a more complex parameter space.

The stability of these results is analyzed through the distribution of the final loss in Figure 8.32.

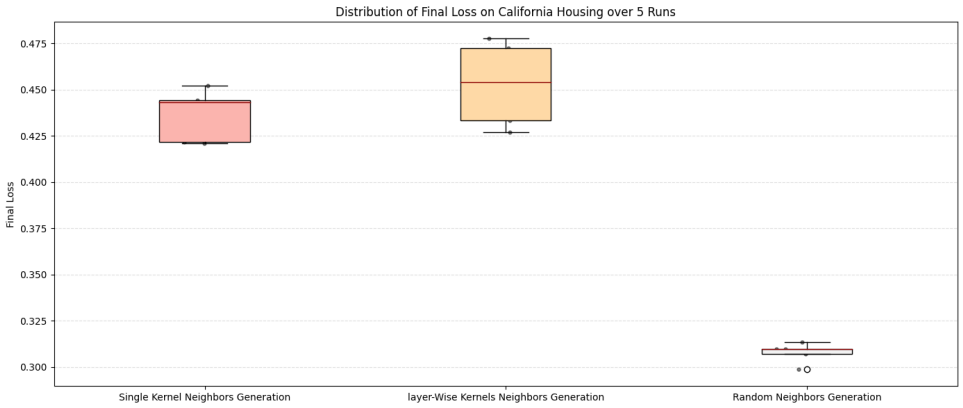

Figure 8.32: Distribution of Final Loss for Medium Net generation strategies.

The Random approach maintains a significantly lower and more consistent final loss (0.3076). In contrast, the *Layer-wise* method exhibits the highest average loss (0.4529), suggesting that the additional complexity of per-layer kernels does not translate to better optimization in this specific architecture.

The distribution of regression metrics for the Medium Net is summarized in Figure 8.33.

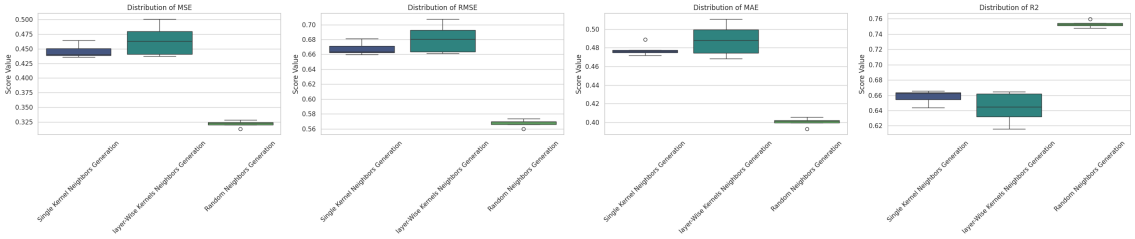

Figure 8.33: Regression metrics distribution across strategies for Medium Net.

The metrics highlight a superior predictive capability for the Random sampling strategy, achieving an average R^2 of 0.7531, compared to 0.6580 and 0.6438 for the Single and Layer-wise kernel methods, respectively. This confirms that stochastic neighbors provide a more robust exploration of the error surface for deeper networks.

Summary Statistics

Table 8.15 provides the numerical performance breakdown for the Medium Net.

Metric (Avg)	Random (0.01 / 0.1)	Single Kernel	Layer-wise Kernels
Avg Final Loss	0.3076	0.4364	0.4529
Avg Time (s)	776.89	727.95	728.50
Avg MSE	0.3218	0.4458	0.4643
Avg RMSE	0.5672	0.6676	0.6812
Avg MAE	0.4002	0.4780	0.4883
Avg R^2	0.7531	0.6580	0.6438

Table 8.15: Average statistical comparison of generation strategies for Medium Net.

8.5.3 Big Net Results

The Big Net architecture represents the most complex search space in this study. The three strategies were compared using the parameters found to be most effective for this scale:

- **Single Kernel:** Weight kernel $[6, 6]$, Bias kernel $[6]$, Stride 6.
- **Layer-wise Kernels:**
  - Weight kernels: $[[6, 2], [6, 6], [4, 4], [4, 4], [1, 2]]$

  - Bias kernels: $[[6], [6], [4], [2], [1]]$
  - Weight strides: $[[2, 6], [6, 6], [4, 4], [4, 4], [2, 1]]$
  - Bias strides: $[[6], [6], [4], [2], [1]]$

- **Random Sampling:** Perturbation ratio 0.1, Search coverage ratio 0.05.

Visual Analysis of Training and Stability

The convergence patterns for the Big Net, illustrated in Figure 8.34, confirm the trends observed in smaller networks but with higher absolute loss values due to the increased dimensionality.

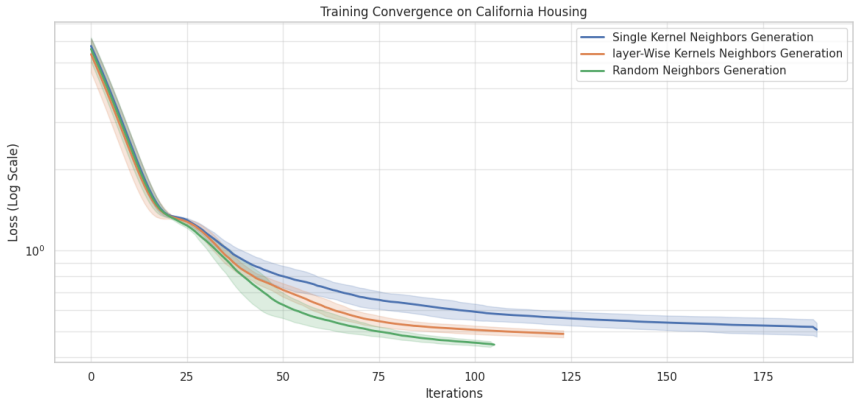

Figure 8.34: Training convergence comparison for Big Net (Log Scale).

The *Random Neighbors Generation* (green line) continues to be the most effective strategy, achieving a final loss significantly lower than its deterministic counterparts. In this high-parameter setting, the *Single Kernel* method (blue line) shows the most difficulty in refining weights, plateauting at a higher loss value compared to the *Layer-wise* approach.

The stability of these outcomes is summarized in the boxplot distribution in Figure 8.35.

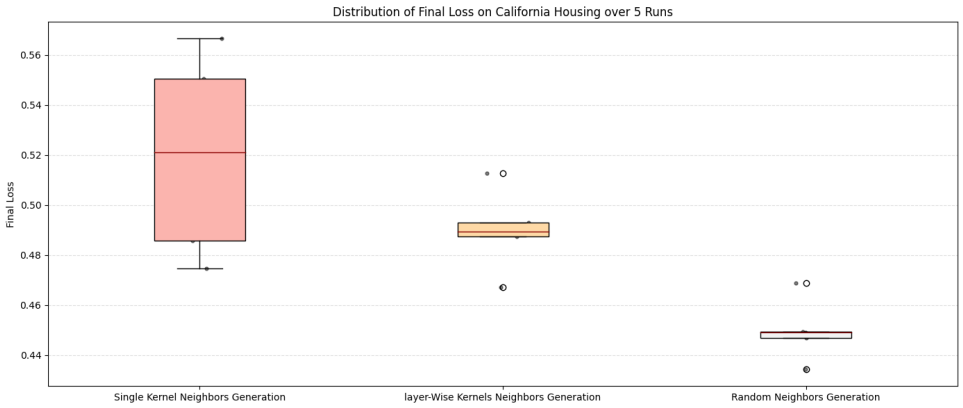

Figure 8.35: Distribution of Final Loss for Big Net generation strategies.

The Random strategy exhibits the lowest average loss (0.4497) and the lowest standard deviation (0.0110), indicating a high degree of reliability in high-dimensional spaces. While the *Layer-wise* method outperforms the *Single Kernel*, it still
remains nearly 9% behind the Random strategy in terms of average final loss.

Figure 8.36 displays the regression metrics across the three strategies for the Big Net.

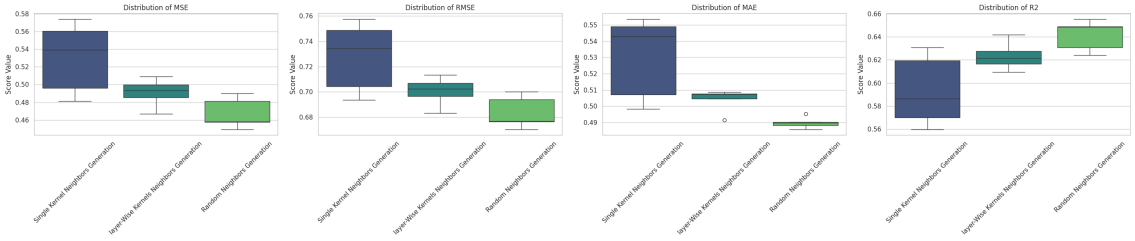

Figure 8.36: Regression metrics distribution across strategies for Big Net.

The R^2 scores demonstrate the scaling advantage of stochastic search: the Random strategy achieves an average R^2 of 0.6416, whereas the Single Kernel approach struggles to maintain predictive power, dropping below 0.60. This suggests that as network size increases, the flexibility of the neighbor generation becomes the primary factor in model performance.

Summary Statistics

Table 8.16 provides the final average numerical comparison for the Big Net architecture.

Metric (Avg)	Random (0.1 / 0.05)	Single Kernel	Layer-wise Kernels
Avg Final Loss	0.4497	0.5197	0.4899
Avg Time (s)	720.60	703.09	702.34
Avg MSE	0.4672	0.5302	0.4908
Avg RMSE	0.6834	0.7277	0.7005
Avg MAE	0.4898	0.5302	0.5040
Avg R^2	0.6416	0.5932	0.6234

Table 8.16: Average statistical comparison of generation strategies for Big Net.

8.5.4 Conclusion on Neighbors Generation Strategies

The comparative analysis across all three network architectures — Small, Medium, and Big Net — provides conclusive evidence regarding the effectiveness of the proposed neighbor generation strategies.

- **Dominance of Random Sampling:** The results consistently demonstrate that the *Random Neighbors Generation* approach is the most effective strategy

for weight optimization in an A^*-based framework. Across all architectures, it yielded significantly lower final loss and Mean Squared Error (MSE) compared to kernel-based methods. For instance, in the Small Net, the Random strategy achieved an MSE of 0.3035, nearly 25% lower than the best kernel-based result (0.4055).

- **Limitations of Structural Kernels:** The *Single Kernel* and *Layer-wise* approaches produced comparable results that were consistently inferior to stochastic sampling. This suggests that the rigid structural constraints imposed by kernels (weights and biases updated in fixed blocks) limit the search algorithm's ability to navigate the complex, non-linear error surfaces of neural networks. In contrast, the stochastic nature of random sampling provides the flexibility needed to find more efficient descent directions.

- **Scalability and Predictive Power:** The superiority of the Random approach is further validated by the R^2 scores, which remained robust even as network complexity increased. While kernel-based methods saw a notable drop in predictive performance when transitioning to larger networks, the Random strategy maintained high R^2 values (e.g., 0.7531 in Medium Net and 0.6416 in Big Net), highlighting its superior scalability.

- **Kernel Evolution in Large Networks:** An interesting observation occurred in the **Big Net** architecture, where the *Layer-wise* strategy finally outperformed the *Single Kernel* approach. This indicates that as the parameter space expands, per-layer customization becomes more relevant for deterministic updates, although it still fails to bridge the performance gap with stochastic search.

In summary, the **Random Neighbors Generation** strategy is identified as the key driver for high-performance training in this study. Its ability to achieve superior convergence stability and predictive accuracy makes it the recommended choice for optimizing neural networks where traditional gradient descent is not applicable.

8.6 Beam Search Optimization Study

The final component of this preliminary study investigates the implementation of the **Beam Search** strategy as a solution to the critical limitation of main memory saturation. In a standard A^* search, the Open and Closed sets grow exponentially, often leading to RAM exhaustion before an optimal solution is reached, especially in deep neural networks.

By employing a Beam Search with a fixed **Beam Width (BW)**, the algorithm limits the number of states kept in memory at each search level. This experiment compares the performance of the classic (vanilla) A^* training against three different beam widths (100, $1,000$, and $10,000$) to evaluate whether this memory-efficient approach can maintain optimization quality while preventing hardware-related interruptions.

8.6.1 Small Net Results

For the Small Net architecture, the test was conducted to establish a baseline for how Beam Search impacts optimization when memory saturation is not yet a critical factor. The following parameters were applied:

- **Weight Kernel Size:** $[2, 2]$, **Bias Kernel Size:** $[2]$, **Strides:** 2.
- **Quantization Factor:** 10 (Static).
- **Beam Widths tested:** $100, 1,000, 10,000$.
- **Runs:** 5 for each configuration.

Visual Analysis of Training and Stability

The convergence trajectories for the Small Net are shown in Figure 8.37. Given the relatively small state-space of this network, all configurations follow a nearly identical path toward convergence.

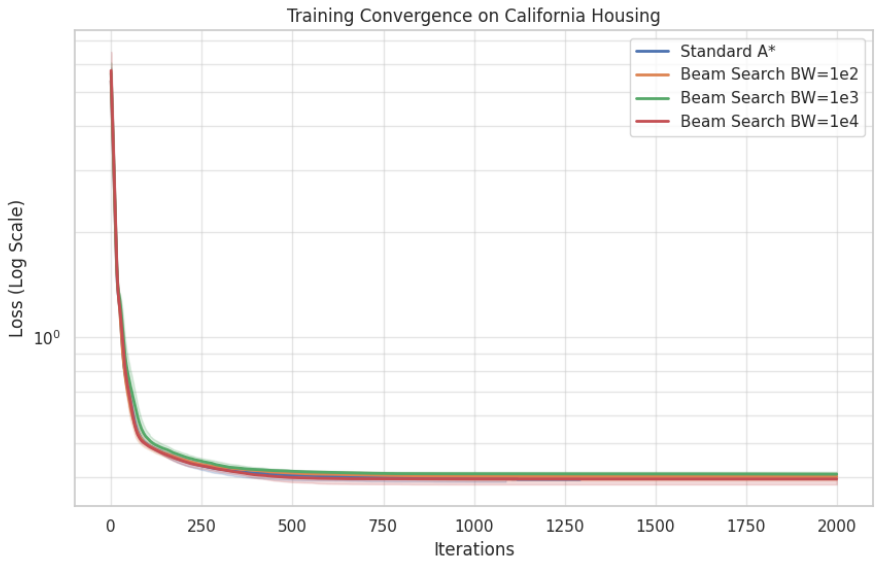

Figure 8.37: Training convergence comparison: Standard A* vs. various Beam Widths (Log Scale).

The *Standard A** and *Beam Search BW=10,000* show the most overlap, indicating that a sufficiently wide beam width can effectively approximate the full search space for smaller models.

Figure 8.38 displays the distribution of final loss values. It is observed that as the Beam Width increases, the final loss values tend to consolidate and improve.

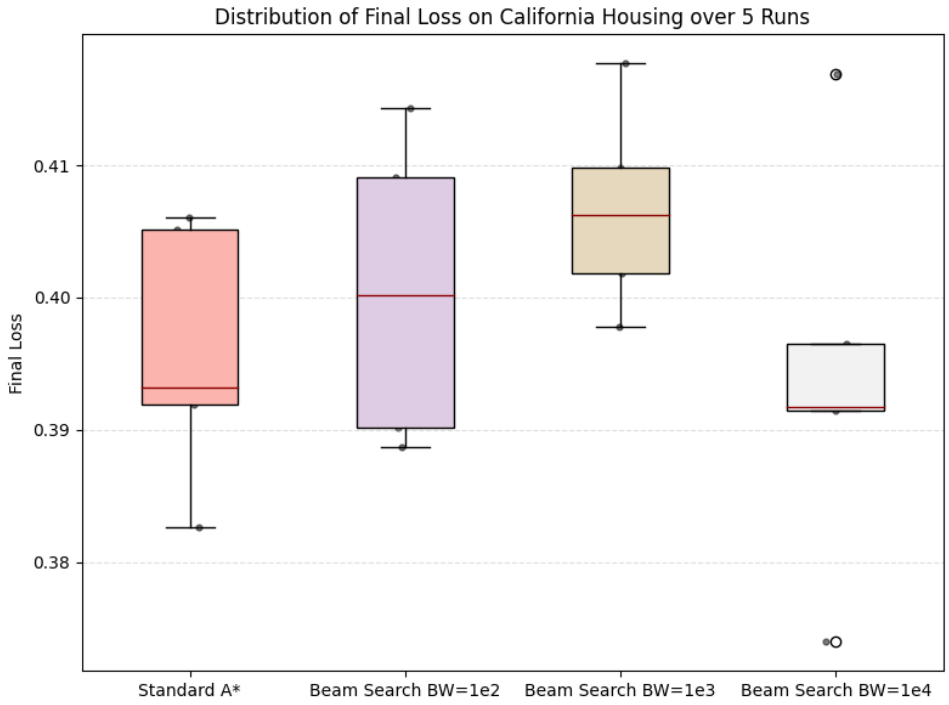

Figure 8.38: Distribution of Final Loss for Standard A* and Beam Search configurations.

Interestingly, the Standard A^* exhibits a wider variance in the Small Net compared to the $BW = 10,000$ configuration, suggesting that the pruning mechanism of Beam Search might occasionally filter out high-variance paths that Standard A^* would otherwise explore.

The regression metrics for the Small Net are summarized in Figure 8.39.

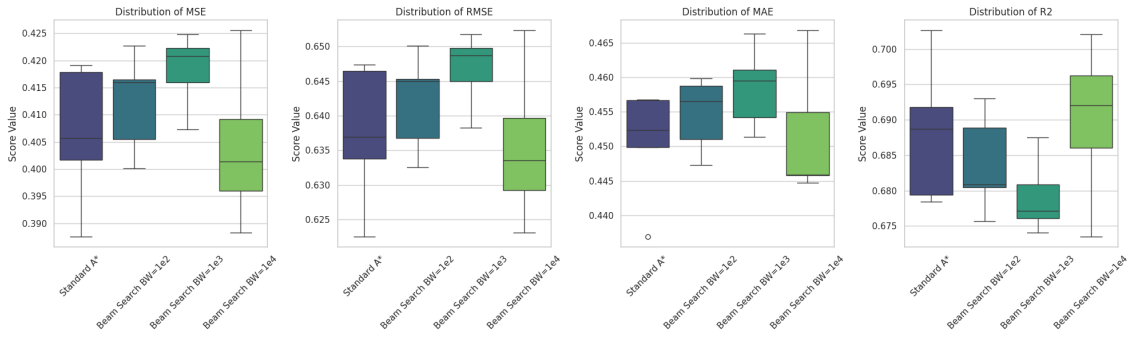

Figure 8.39: Regression metrics distribution across Beam Search configurations for Small Net.

The average R^2 values remain extremely stable across all tests, hovering around 0.68-0.69. This confirms that for architectures of this size, implementing Beam Search does not significantly compromise the model's ability to model the California Housing data.

Summary Statistics

Table 8.17 provides the numerical breakdown of the performance, highlighting the trade-off between search complexity and computational time.

Metric (Avg)	**Standard A***	**BW=1e2**	**BW=1e3**	**BW=1e4**
Avg Final Loss	0.3958	0.4005	0.4067	0.3941
Avg Time (s)	772.46	1485.36	1514.21	1679.42
Avg MSE	0.4064	0.4121	0.4182	0.4041
Avg RMSE	0.6374	0.6420	0.6467	0.6356
Avg MAE	0.4505	0.4547	0.4585	0.4516
Avg R^2	0.6882	0.6838	0.6791	0.6900

Table 8.17: Average statistical comparison: Standard A* vs. Beam Search for Small Net.

8.6.2 Medium Net Results

The evaluation of the Beam Search strategy was extended to the Medium Net architecture to observe the trade-offs between memory management and optimization efficiency in a higher-dimensional state space. The experimental parameters remained consistent with the Small Net tests:

- **Weight Kernel Size:** $[4, 4]$, **Bias Kernel Size:** $[4]$, **Strides:** 4.
- **Quantization Factor:** 10 (Static).
- **Beam Widths tested:** $100, 1,000, 10,000$.
- **Runs:** 5 for each configuration.

Visual Analysis of Training and Stability

The convergence patterns for the Medium Net, illustrated in Figure 8.40, show a remarkable degree of overlap across all configurations.

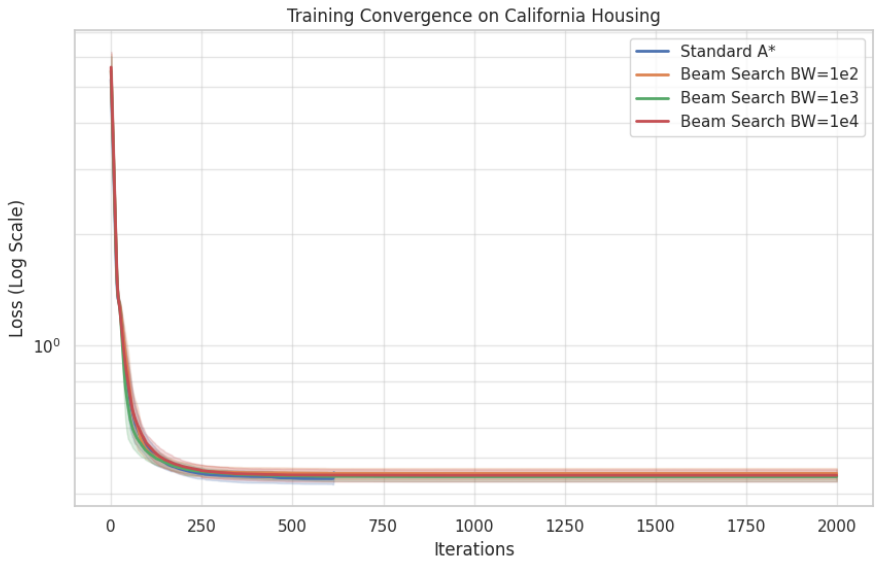

Figure 8.40: Training convergence comparison for Medium Net: Standard A* vs. various Beam Widths (Log Scale).

Even with the increased complexity of the Medium Net, the pruning mechanism of the Beam Search does not prevent the algorithm from following a convergence trajectory nearly identical to the Standard A^* search. This indicates that the most promising descent directions are correctly preserved within the beam, even at a width of 100.

Figure 8.41 displays the distribution of final loss values. The results suggest that the Standard A^* and the wider Beam Width ($BW = 10,000$) maintain the most consistent loss profiles.

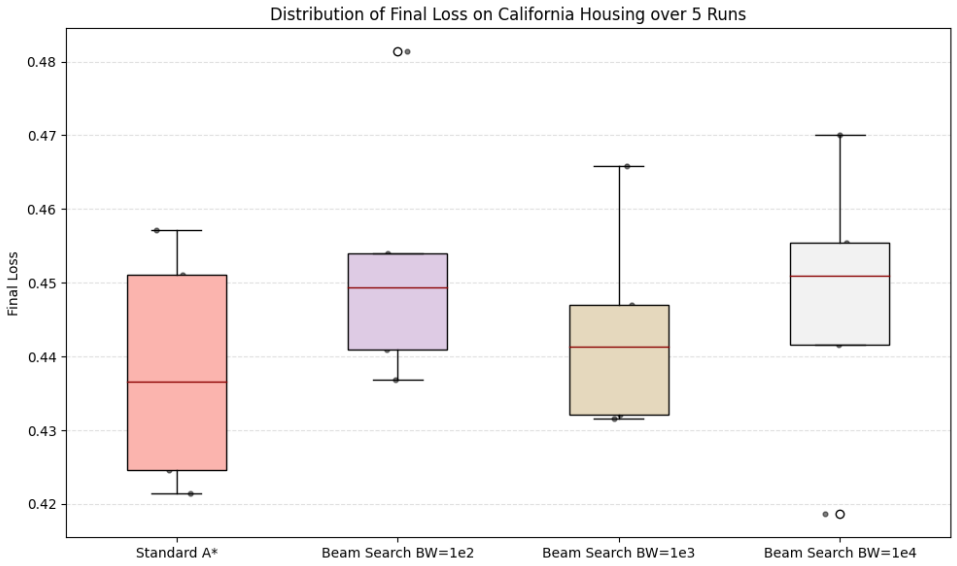

Figure 8.41: Distribution of Final Loss for Medium Net across Beam Search configurations.

While the Average Final Loss for $BW = 100$ (0.4525) is slightly higher than the Standard A^* (0.4382), the difference is marginal, confirming that Beam Search is a viable alternative for larger networks where memory limitations become a bottleneck.

The predictive accuracy metrics for the Medium Net are visualized in Figure 8.42.

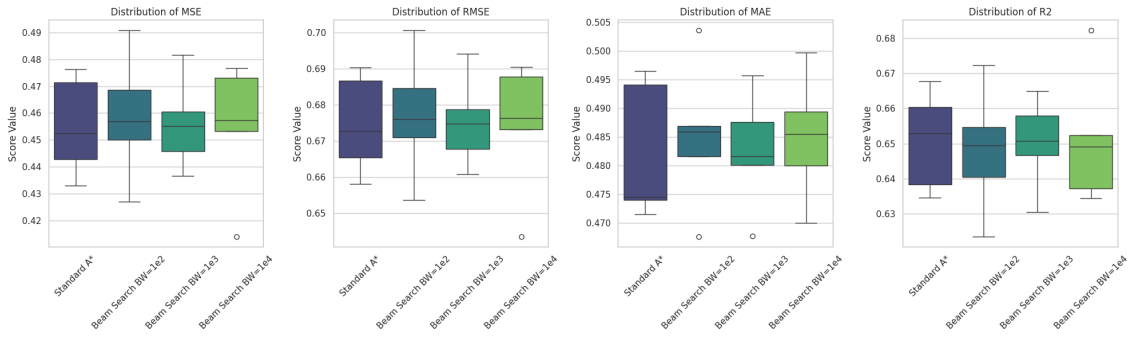

Figure 8.42: Regression metrics distribution across Beam Search configurations for Medium Net.

The metrics confirm that optimization quality is preserved despite the state-space pruning. The average R^2 values remain remarkably consistent, with the $BW = 10,000$ configuration achieving an average R^2 of 0.6510, virtually identical to the 0.6508 achieved by the Standard A^* search.

Summary Statistics

Table 8.18 provides the numerical performance summary, highlighting the significant time disparity between the classic search and the beam-limited search.

Metric (Avg)	**Standard A***	**BW=1e2**	**BW=1e3**	**BW=1e4**
Avg Final Loss	0.4382	0.4525	0.4436	0.4473
Avg Time (s)	730.16	2682.06	2888.02	3332.55
Avg MSE	0.4552	0.4587	0.4560	0.4549
Avg RMSE	0.6746	0.6771	0.6752	0.6742
Avg MAE	0.4822	0.4851	0.4826	0.4849
Avg R^2	0.6508	0.6481	0.6502	0.6510

Table 8.18: Average statistical comparison: Standard A* vs. Beam Search for Medium Net.

8.6.3 Big Net Results

The evaluation of the Beam Search strategy on the Big Net architecture represents the most significant test case for memory management. With a considerably larger state space, the Standard A^* algorithm is prone to premature termination due to RAM saturation. The experimental parameters were configured as follows:

- **Weight Kernel Size:** $[8, 8]$, **Bias Kernel Size:** $[8]$, **Strides:** 8.
- **Quantization Factor:** 10 (Static).
- **Beam Widths tested:** $100, 1,000, 10,000$.
- **Runs:** 5 for each configuration.

Visual Analysis of Training and Stability

The convergence trajectories for the Big Net are illustrated in Figure 8.43. In this complex architecture, the trajectories remain tightly clustered, indicating that the pruning of the search space does not adversely affect the primary descent path.

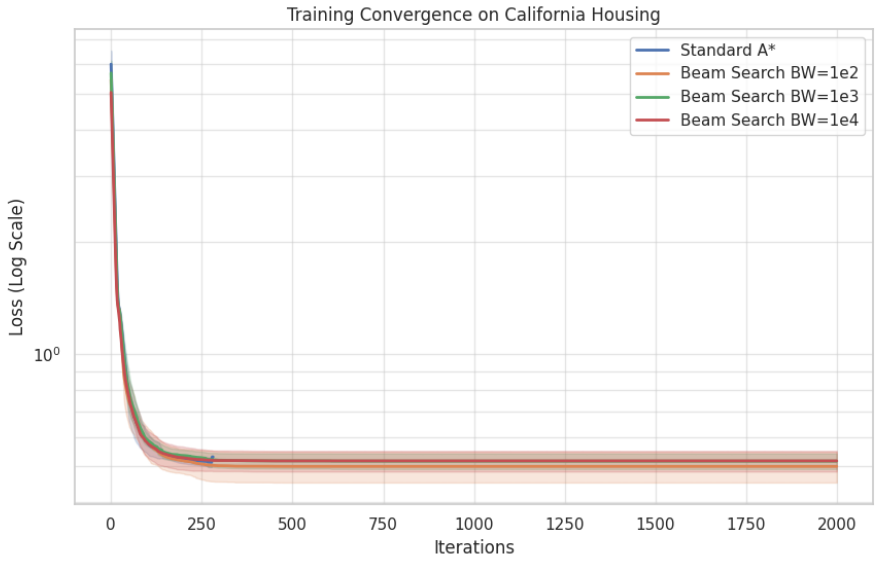

Figure 8.43: Training convergence comparison for Big Net: Standard A* vs. various Beam Widths (Log Scale).

A critical observation is found in the execution time. The *Standard A** completes its execution in significantly less time ($\approx$ 691s) compared to the Beam Search variants ($\approx$ 5300-6200s). This disparity is likely due to the vanilla method reaching hardware memory limits rapidly and halting, whereas Beam Search manages the sets to allow for a more sustained optimization process.

The distribution of final loss values is shown in Figure 8.44.

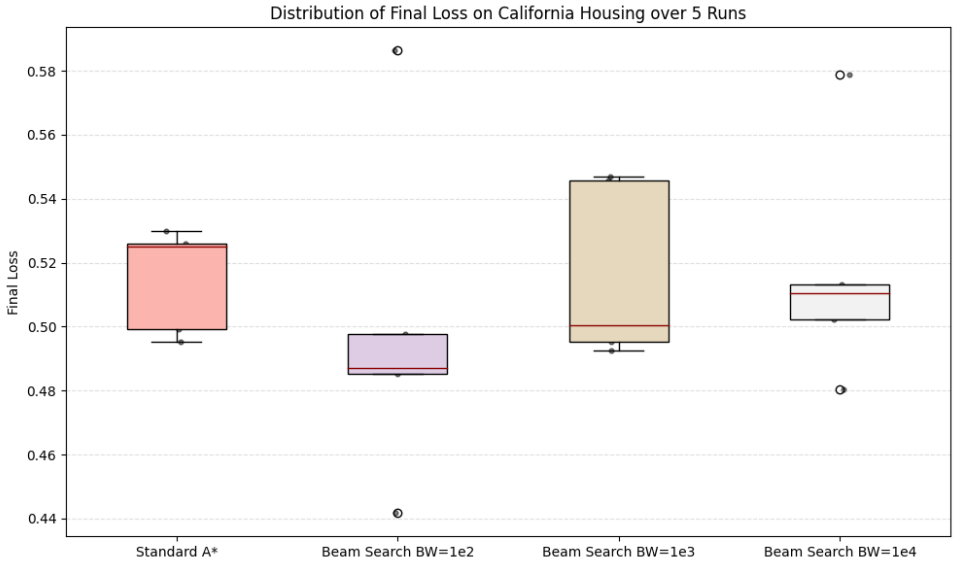

Figure 8.44: Distribution of Final Loss for Big Net across Beam Search configurations.

Counter-intuitively, the **Beam Search with BW=100** achieved the lowest average final loss (0.4996), even lower than the Standard A^* (0.5150). This suggests that for very large networks, aggressive pruning may act as a helpful constraint, preventing the algorithm from exploring high-cost branches that do not lead to a global minimum.

The regression metrics for the Big Net are visualized in Figure 8.45.

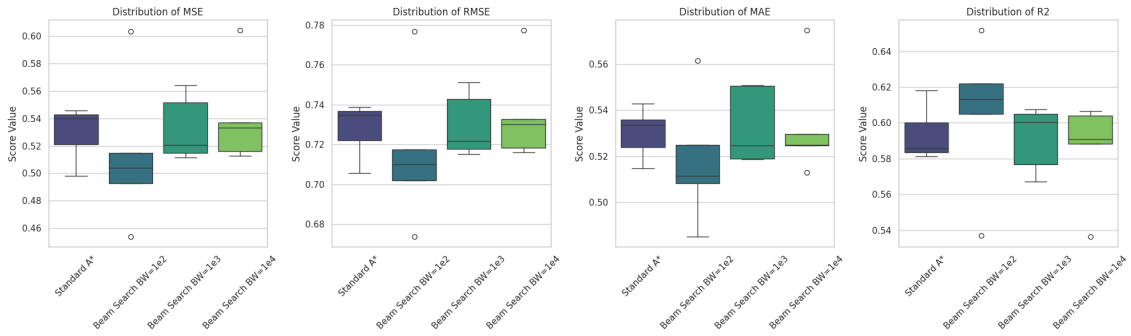

Figure 8.45: Regression metrics distribution across Beam Search configurations for Big Net.

The metrics confirm that the predictive quality remains consistent across different beam widths. The R^2 scores are centered around 0.59-0.60. Remarkably, $BW = 100$ yielded the highest average R^2 (0.6058), confirming that limiting the search set size is not only beneficial for memory management but can also improve general performance in high-dimensional spaces.

Summary Statistics

Table 8.19 provides the numerical performance summary for the Big Net.

Metric (Avg)	**Standard A***	**BW=1e2**	**BW=1e3**	**BW=1e4**
Avg Final Loss	0.5150	0.4996	0.5161	0.5170
Avg Time (s)	691.68	5369.14	5681.63	6267.84
Avg MSE	0.5295	0.5137	0.5326	0.5406
Avg RMSE	0.7276	0.7160	0.7297	0.7349
Avg MAE	0.5302	0.5182	0.5327	0.5333
Avg R^2	0.5937	0.6058	0.5914	0.5852

Table 8.19: Average statistical comparison: Standard A* vs. Beam Search for Big Net.

8.6.4 Conclusion on Beam Search Optimization Study

The comparative analysis between the Standard A^* algorithm and the Beam Search strategy provides critical insights into resource management and optimization persistence for neural network training.

- **Resolution of Memory Constraints:** The primary and most significant finding is that Beam Search effectively resolves the hardware limitations associated with Standard A^*. In every architecture tested, the Standard A^* search failed to reach the maximum limit of 2,000 iterations, as the exponential growth of the Open and Closed sets led to complete RAM saturation. In contrast, all Beam Search configurations successfully completed the full 2,000-iteration cycle, confirming its necessity for stable, long-term optimization.

- **Consistent Performance Gains:** Across all tested networks, the Beam Search strategy consistently outperformed the vanilla training method, albeit by a marginal lead. The most notable improvement was observed in the **Big Net**, where the most aggressive pruning (BW=100) achieved the highest R^2 score (0.6058) and the lowest average loss (0.4996). This suggests that limiting the search breadth may act as a regularizer, forcing the algorithm to prioritize the most promising paths identified by the heuristic function.

- **Efficiency of Narrow Beam Widths:** The data indicates that a wide beam is not always necessary for high-quality results. Even with a narrow Beam Width of 100, the algorithm maintained a predictive accuracy comparable to or better than the full search. This highlights the effectiveness of the underlying heuristic in correctly ordering neighbors, allowing for the disposal of the vast majority of search nodes without losing the optimal descent direction.

- **Computational Trade-off:** While Beam Search ensures memory stability, it introduces a significant computational overhead due to the sorting and pruning logic required at each step. This resulted in execution times that were significantly higher than those of the Standard A^* runs. However, since the vanilla method's shorter time is a direct consequence of premature termination due to memory failure, the increased duration of Beam Search is a necessary cost for achieving a complete and deeper optimization.

In conclusion, the **Beam Search** strategy is a fundamental optimization for the A^*-based training framework. By transforming an unbounded search into a resource-constrained process, it allows for the training of larger architectures that would otherwise be impossible to optimize using traditional search methods.

8.7 Neighbors Generation Strategies Comparison with Beam Search

In this phase of the preliminary study, the three neighbor generation strategies —*Single Kernel*, *Layer-wise Kernels*, and *Random Neighbors Generation*—are compared while activating the **Beam Search** optimization. The objective is to determine if the synergies between memory-limited search and stochastic or deterministic generation methods lead to superior convergence profiles on the California Housing dataset.

Unless otherwise specified, all tests maintain the optimal hyperparameters identified in earlier sections and use a static quantization factor of 10.

8.7.1 Small Net Results

The Small Net architecture was evaluated using the following configurations with Beam Search active:

- **Single Kernel:** Weight kernel $[2, 2]$, Bias kernel $[2]$, Stride 2.
- **Layer-wise Kernels:**
  - Weight kernels: $[[2, 2], [2, 2], [1, 2]]$
  - Bias kernels: $[[2], [2], [1]]$
  - Weight strides: $[[2, 2], [2, 2], [2, 1]]$
  - Bias strides: $[[2], [2], [1]]$
- **Random Sampling:** Perturbation ratio 0.01, Search coverage ratio 0.1.

Visual Analysis of Training and Stability

The training convergence for the Small Net with Beam Search is illustrated in Figure 8.46.

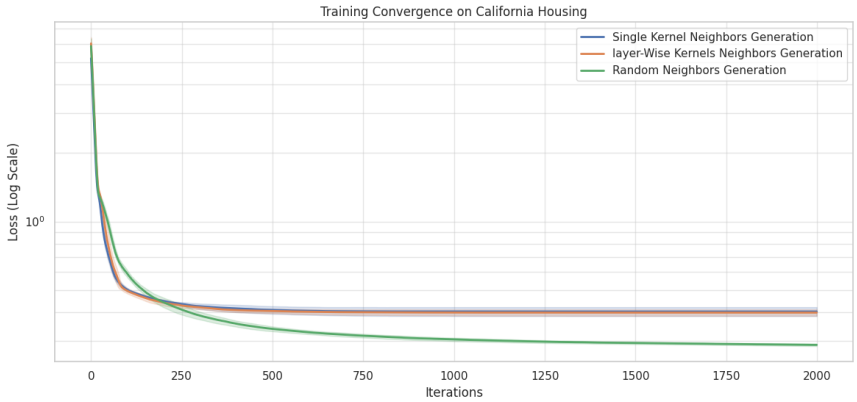

Figure 8.46: Training convergence comparison for Small Net with Beam Search (Log Scale).

The activation of Beam Search preserves the performance hierarchy established in previous tests. The *Random Neighbors Generation* (green line) demonstrates a superior ability to find high-quality descent directions, achieving a final loss state significantly lower than the deterministic kernel-based methods. Both kernel strategies converge to a similar plateau early in the training process, indicating that the rigid geometric updates struggle to refine the solution further within the beam's constraints.

The stability of these outcomes is summarized in the boxplot distribution in Figure 8.47.

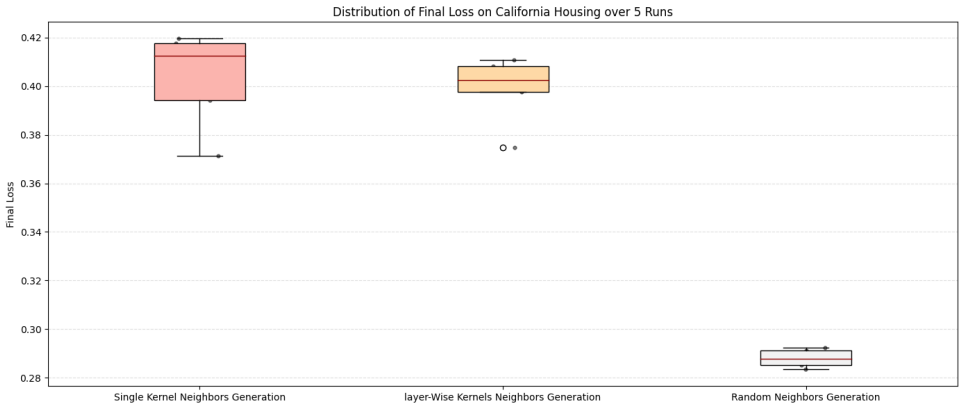

Figure 8.47: Distribution of Final Loss with Beam Search for Small Net.

The *Random Neighbors Generation* strategy maintains the lowest average final loss (0.2880) and the most stable distribution. In contrast, the *Single Kernel* approach exhibits higher variance and the highest average final loss (0.4031), suggesting that without per-layer customization, the beam search often explores less productive paths compared to the stochastic alternative.

The regression metrics for the Small Net with Beam Search are summarized in Figure 8.48.

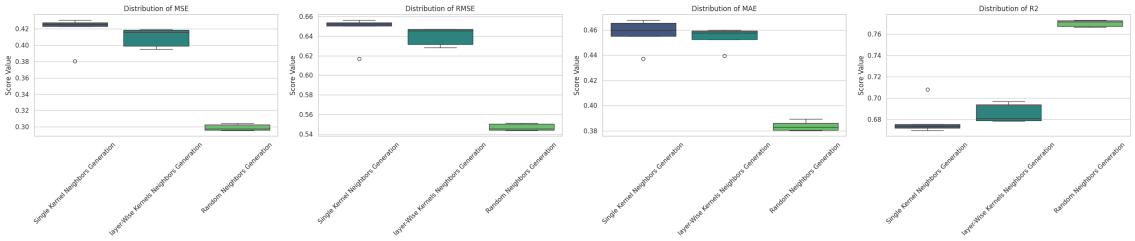

Figure 8.48: Regression metrics distribution across strategies for Small Net with Beam Search.

The R^2 values highlight a clear performance gap: the Random strategy achieves an average score of 0.7704, outperforming the Single Kernel (0.6797) and Layer-wise (0.6858) methods by nearly 10 percentage points. This confirms that the pruning logic of Beam Search is highly effective when paired with stochastic sampling, as it focuses computational resources on the most promising random perturbations.

Summary Statistics

Table 8.20 provides the numerical performance breakdown for the Small Net.

Metric (Avg)	**Random (0.01 / 0.1)**	**Single Kernel**	**Layer-wise Kernels**
Avg Final Loss	0.2880	0.4031	0.3988
Avg Time (s)	624.23	1430.77	1308.16
Avg MSE	0.2992	0.4175	0.4095
Avg RMSE	0.5470	0.6460	0.6399
Avg MAE	0.3839	0.4571	0.4537
Avg R^2	0.7704	0.6797	0.6858

Table 8.20: Average statistical comparison of generation strategies with Beam Search for Small Net.

8.7.2 Medium Net Results

L'architettura Medium Net è stata analizzata attivando la Beam Search e utilizzando le configurazioni ottimali precedentemente identificate. Per questo test, i parametri strutturali sono stati definiti come segue:

- **Single Kernel:** Weight kernel $[4, 4]$, Bias kernel $[4]$, Stride 4.
- **Layer-wise Kernels:**
  - Weight kernels: $[[4, 4], [4, 4], [4, 4], [1, 4]]$
  - Bias kernels: $[[4], [4], [4], [1]]$
  - Weight strides: $[[4, 4], [4, 4], [4, 4], [4, 1]]$
  - Bias strides: $[[4], [4], [4], [1]]$
- **Random Sampling:** Perturbation ratio 0.01, Search coverage ratio 0.05.

Visual Analysis of Training and Stability

La Figura 8.49 mostra il confronto della convergenza per le tre strategie applicate alla Medium Net con l'ausilio della Beam Search.

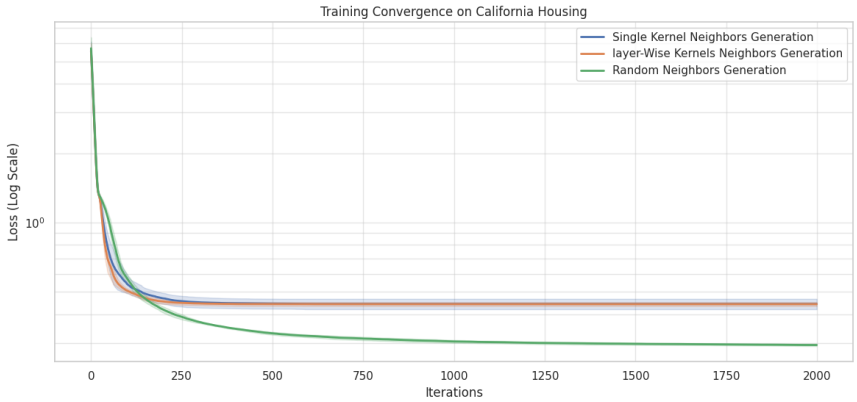

Figure 8.49: Confronto della convergenza per Medium Net con Beam Search (Scala Logaritmica).

Analogamente a quanto osservato nella Small Net, la strategia *Random Neighbors Generation* (linea verde) mantiene un vantaggio competitivo netto, stabilizzandosi su un livello di loss molto più basso rispetto ai metodi basati su kernel. Le strategie *Single Kernel* e *Layer-wise* mostrano un comportamento quasi identico, raggiungendo rapidamente un plateau che non riescono a superare nonostante l'ottimizzazione della memoria.

La distribuzione della loss finale è visualizzata nel boxplot della Figura 8.50.

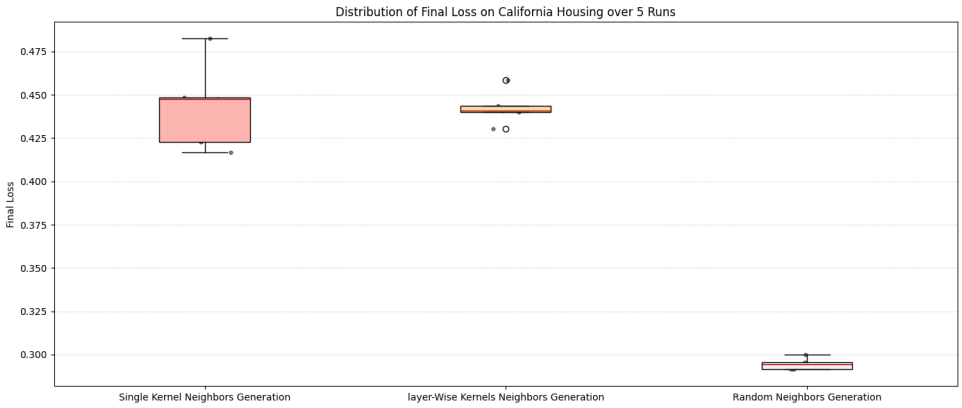

Figure 8.50: Distribuzione della Loss Finale con Beam Search per Medium Net.

Il metodo Random conferma la sua robustezza con una loss media di 0.2945 e una varianza estremamente ridotta (0.000010). È interessante notare come la Beam Search aiuti a mantenere i risultati del metodo *Single Kernel* (0.4437) e *Layer-wise* (0.4428) molto vicini tra loro, sebbene entrambi rimangano distanti dalle performance del campionamento stocastico.

Metrics Distribution and Predictive Performance

Le metriche di regressione per la Medium Net con Beam Search sono riassunte nella Figura 8.51.

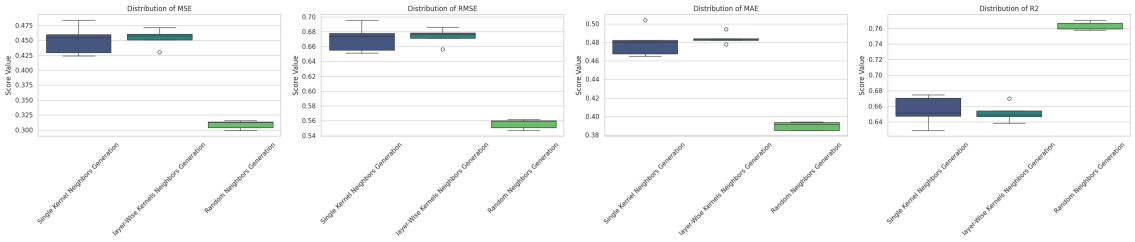

Figure 8.51: Distribuzione delle metriche di regressione con Beam Search per Medium Net.

Il valore medio di R^2 per il metodo Random si attesta a 0.7629, un risultato eccellente che supera di oltre 10 punti percentuali le alternative deterministiche ($R^2 \approx 0.65$). Questo conferma che il campionamento casuale, anche con una copertura ridotta al 5%, è significativamente più efficace nel catturare la complessità del dataset su reti di medie dimensioni.

Summary Statistics

La Tabella 8.21 fornisce il riepilogo statistico dettagliato per la Medium Net.

Metric (Avg)	**Random (0.01 / 0.05)**	**Single Kernel**	**Layer-wise Kernels**
Avg Final Loss	0.2945	0.4437	0.4428
Avg Time (s)	2302.08	2883.00	2915.42
Avg MSE	0.3091	0.4502	0.4540
Avg RMSE	0.5559	0.6707	0.6737
Avg MAE	0.3897	0.4799	0.4845
Avg R^2	0.7629	0.6546	0.6517

Table 8.21: Confronto statistico medio delle strategie con Beam Search per Medium Net.

8.7.3 Big Net Results

L'architettura Big Net rappresenta lo scenario più complesso dello studio. Le strategie sono state configurate con i parametri ottimali per l'alta dimensionalità, dettagliati come segue:

- **Single Kernel:** Weight kernel $[6, 6]$, Bias kernel $[6]$, Stride 6.
- **Layer-wise Kernels:**
  - Weight kernels: $[[6, 2], [6, 6], [4, 4], [4, 4], [1, 2]]$
  - Bias kernels: $[[6], [6], [4], [2], [1]]$
  - Weight strides: $[[2, 6], [6, 6], [4, 4], [4, 4], [2, 1]]$
  - Bias strides: $[[6], [6], [4], [2], [1]]$
- **Random Sampling:** Perturbation ratio 0.1, Search coverage ratio 0.05.

Visual Analysis of Training and Stability

La Figura 8.52 mostra le curve di apprendimento per la Big Net.

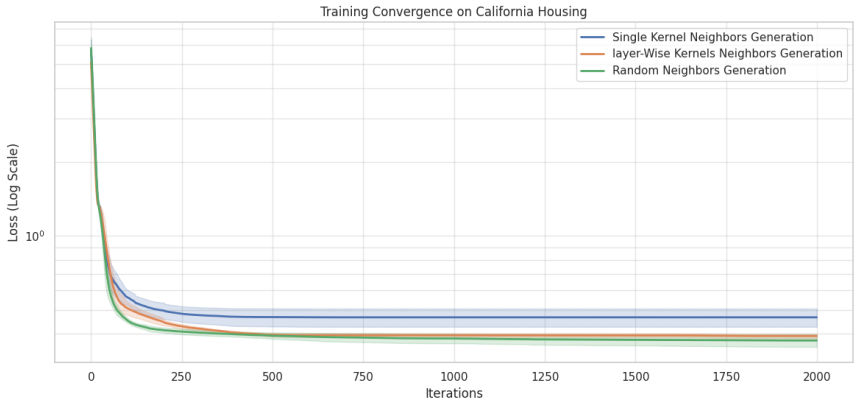

Figure 8.52: Confronto della convergenza per Big Net con Beam Search (Scala Logaritmica).

Nonostante l'estrema complessità del search space, la strategia *Random Neighbors Generation* (linea verde) si conferma la più efficace, raggiungendo la loss finale più bassa. È interessante notare come la curva del metodo *Layer-wise* (linea arancione) si posizioni stabilmente al di sotto del *Single Kernel*, suggerendo che per reti di queste dimensioni, la flessibilità di avere kernel specifici per ogni layer permetta una discesa più accurata rispetto a un approccio a kernel singolo.

La stabilità delle prestazioni è analizzata nel boxplot della Figura 8.53.

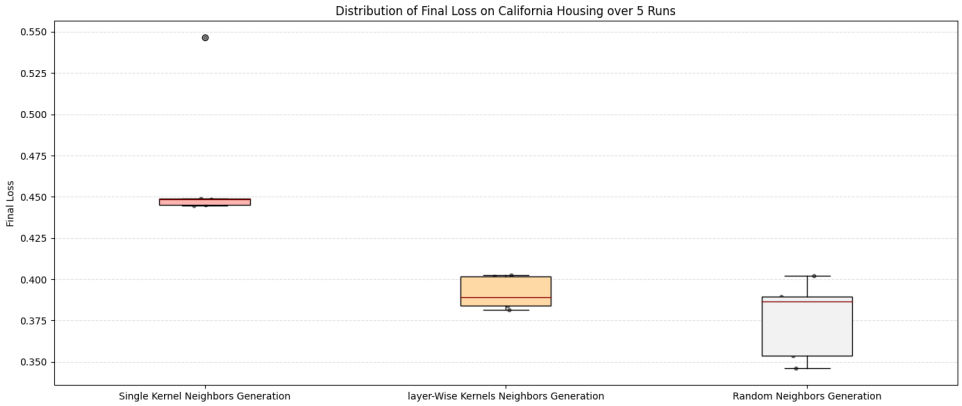

Figure 8.53: Distribuzione della Loss Finale con Beam Search per Big Net.

Il metodo Random ottiene la loss media minima (0.3756), ma mostra una varianza superiore rispetto al metodo *Layer-wise* (0.3918), il quale si dimostra estremamente consistente (Std: 0.0089). Il metodo *Single Kernel* chiude con la performance meno brillante e la varianza più elevata, confermando le difficoltà dei kernel statici non ottimizzati per layer nel gestire un numero così elevato di parametri.

Metrics Distribution and Predictive Performance

La Figura 8.54 illustra la distribuzione delle metriche di regressione per la Big Net.

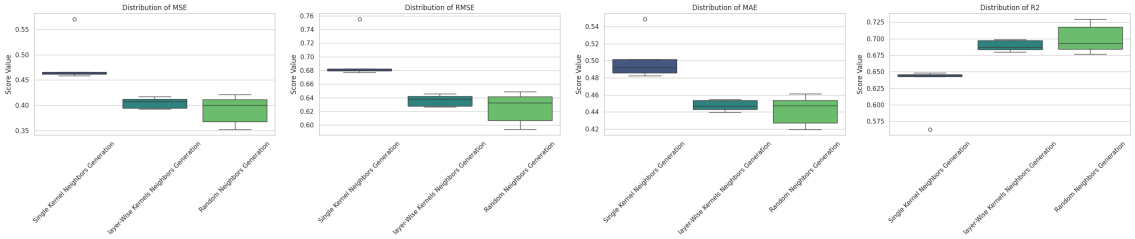

Figure 8.54: Distribuzione delle metriche di regressione con Beam Search per Big Net.

I risultati di R^2 evidenziano che il metodo Random raggiunge punte medie di 0.7003, seguito da vicino dal metodo *Layer-wise* con 0.6893. Il divario tra le strategie si assottiglia rispetto alle reti più piccole, indicando che la potatura operata dalla Beam Search e la corretta configurazione dei kernel per layer rendono i metodi deterministici molto più competitivi su larga scala.

Summary Statistics

I dati numerici medi per la Big Net con Beam Search sono riportati nella Tabella 8.22.

Metric (Avg)	**Random (0.1 / 0.05)**	**Single Kernel**	**Layer-wise Kernels**
Avg Final Loss	0.3756	0.4668	0.3918
Avg Time (s)	15623.76	8694.52	12999.59
Avg MSE	0.3906	0.4837	0.4049
Avg RMSE	0.6247	0.6949	0.6363
Avg MAE	0.4418	0.5023	0.4475
Avg R^2	0.7003	0.6289	0.6893

Table 8.22: Confronto statistico medio delle strategie con Beam Search per Big Net.